

Projet fil rouge — Rapport

Data Lake — Phases 2 et 3

Industrialisation de l'ingestion et intégration du streaming

MinIO · Python · PySpark · Kafka · Spark Structured Streaming

Binôme : Nour REKIK — Bouchra TYAL

Octobre 2026

1. Présentation du binôme et répartition du travail

Membre	Travail réalisé
Nour REKIK	À compléter
Bouchra TYAL	À compléter

Ce rapport documente l'évolution du Data Lake construit en phase 1. Conformément à la consigne, la plateforme n'est pas reconstruite : elle est auditée (V1), puis enrichie d'une ingestion automatisée (V2) et d'une ingestion streaming (V3). Chaque test est présenté selon le format **objectif → action → preuve → observation → interprétation**.

2. Architecture V1 — Socle Data Lake (phase 1)

Mission 1 — Auditer l'architecture existante. Avant de faire évoluer la plateforme, nous avons recensé l'ensemble de ses composants et identifié ses limites, en particulier celles liées aux interventions manuelles et à l'exploitation.

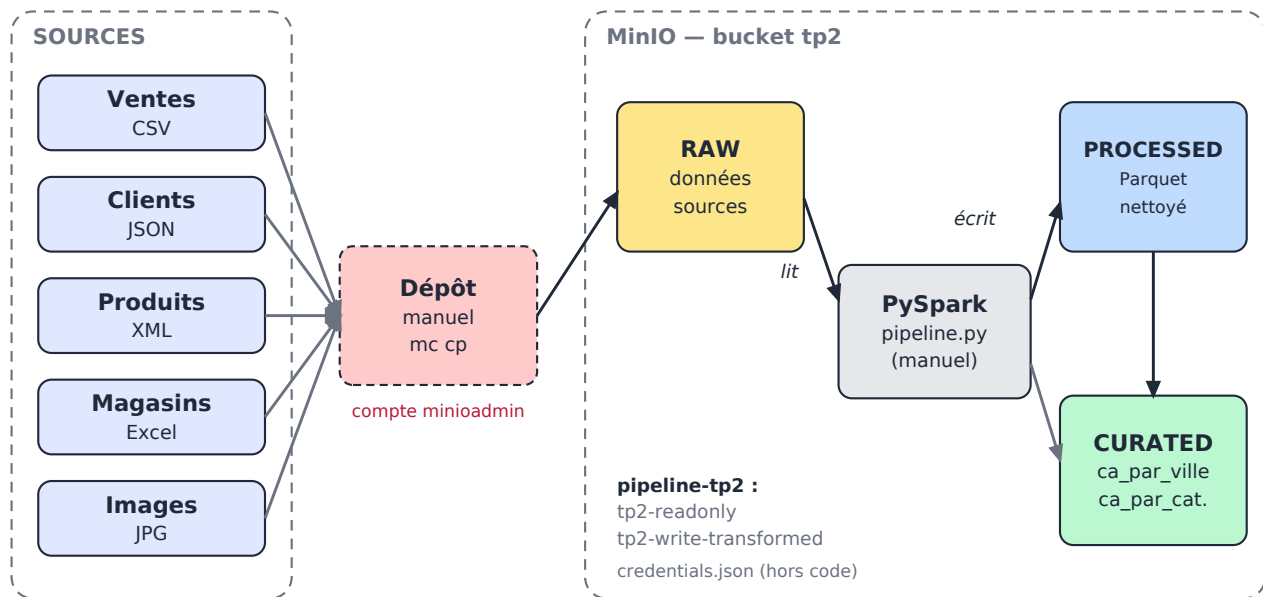


Figure 1 — Architecture V1 — Socle Data Lake

Lecture du schéma. Les cinq sources sont déposées **à la main** dans la zone RAW avec la commande `mc cp`, en utilisant le compte administrateur. Le pipeline PySpark, lancé manuellement par `python src/pipeline.py`, lit RAW avec l'identité `pipeline-tp2`, écrit les données nettoyées en Parquet dans PROCESSED, puis les indicateurs dans CURATED.

2.1 Inventaire des composants

Composant	Réalisation en phase 1
Sources	Ventes (CSV), clients (JSON), produits (XML), magasins (Excel), images produits (JPG)
Stockage	MinIO (API S3 sur le port 9000, console sur le port 9001), bucket tp2
Zones	raw/ (sources d'origine), processed/ (Parquet nettoyé et enrichi), curated/ (indicateurs métier)
Alimentation de RAW	Manuelle : <code>mc cp</code> , un fichier après l'autre, avec le compte administrateur
Scripts Python	<code>config.py</code> (connexion), <code>spark_utils.py</code> (lecture des sources, écriture Parquet sans suppression), <code>test_permissions.py</code> , <code>explore_minio.py</code> , <code>explore_sources.py</code>
Traitements PySpark	<code>pipeline.py</code> : nettoyage, typage, dédoublonnage, Parquet, jointures, montant, CA par ville et par catégorie
Utilisateurs	<code>minioadmin</code> (administrateur) ; <code>pipeline-tp2</code> (application de traitement, avec Access Key)
Politiques	<code>tp2-readonly</code> (ListBucket, GetObject sur tp2) ; <code>tp2-write-transformed</code> (PutObject sur processed/ et curated/)
Credentials	<code>credentials.json</code> à la racine du projet ou variables d'environnement, jamais dans le code ; exclus du rendu

2.2 Limites identifiées

L'audit fait apparaître plusieurs limites, qui justifient les évolutions des phases 2 et 3 :

#	Limite	Conséquence	Réponse prévue
1	Alimentation de RAW manuelle (mc cp)	Non automatisable ; risque d'oubli ou de mauvais préfixe ; dépend d'une intervention humaine	Ingestion automatisée depuis incoming/ (missions 2, 4)
2	Dépôt avec le compte administrateur	Droits excessifs : une erreur peut écraser ou supprimer des données (c'est arrivé pendant le TP1)	Identité dédiée ingestion-user (mission 3)
3	Aucune traçabilité des arrivées	Impossible de savoir quand, quoi et d'où un fichier a été chargé	Journal d'ingestion (mission 6)
4	Aucun contrôle à l'arrivée	Un fichier vide, corrompu ou déjà chargé entre dans RAW sans alerte	Détection des doublons et gestion des erreurs (missions 5, 7)
5	Pipeline lancé à la main	Les données ne sont à jour que si quelqu'un pense à relancer le traitement	Enchaînement ingestion → pipeline (mission 8)
6	Traitement uniquement batch	Les ventes ne sont visibles qu'après une exécution complète, jamais en temps réel	Kafka + Spark Structured Streaming (phase 3)

3. Phase 2 — Industrialiser l'ingestion

La phase 2 supprime le dépôt manuel dans MinIO. Les fichiers arrivent dans une zone locale `incoming/`, un programme Python les contrôle et les dépose dans RAW avec une identité dédiée, puis le pipeline PySpark de la phase 1 est relancé sans être réécrit.

3.1 Mission 2 — Zone d'arrivée externe

Le projet de la phase 1 est d'abord sauvegardé (`tp2_phase1_sauvegarde`), puis la zone `incoming/` est créée avec un sous-dossier par source : `ventes/`, `clients/`, `produits/`, `magasins/`. Cette zone **ne fait pas partie du Data Lake** : elle simule les systèmes externes (caisse, CRM, ERP...) qui déposent leurs exports. Les images produits n'y ont pas de dossier : elles restent dans RAW et ne sont pas traitées.

```
3889633                20 files
(.venv) rekiknour@KHALESSI datalake % cd ~/Desktop/datalake
[cp -R tp2 tp2_phase1_sauvegarde
(.venv) rekiknour@KHALESSI datalake % cd ~/Desktop/datalake/tp2
source .venv/bin/activate
mkdir -p incoming/ventes incoming/clients incoming/produits incoming/magasins
ls incoming
clients      magasins      produits      ventes
(.venv) rekiknour@KHALESSI tp2 % █
```

Figure 2 — Sauvegarde de la phase 1 et création de `incoming/`

3.2 Mission 3 — Séparer ingestion et traitement

Deux identités techniques remplacent l'usage du compte administrateur, selon le principe du **moindre privilège** :

Identité	Politiques	RAW	PROCESSED / CURATED	Responsabilité
ingestion-user	tp2-ingestion-raw (ListBucket sur <code>raw/*</code> , GetObject + PutObject sur <code>raw/*</code>)	Lecture, écriture	Aucun accès	Alimenter RAW
pipeline-user	tp2-readonly + tp2-write-transformed (réutilisées de la phase 1)	Lecture seule	Lecture, écriture	Transformer les données

Aucune identité ne dispose de `s3:DeleteObject`. Chaque identité possède sa propre Access Key ; les secrets sont stockés dans `config/credentials_ingestion.json` et `config/credentials_pipeline.json` (ou en variables d'environnement), exclus du rendu.

Test — Credentials inadaptés et matrice des droits

Objectif

Vérifier que chaque identité ne peut faire que ce qui relève de sa responsabilité, et qu'une identité inadaptée ou des credentials faux sont refusés.

Action

Exécution de `src/ingestion/test_identities.py` : chaque identité tente d'écrire dans RAW, PROCESSED et CURATED, de lire RAW et de supprimer dans RAW. Le script teste ensuite la clé d'ingestion utilisée pour écrire dans CURATED, une mauvaise Secret Key et une Access Key inexistante.

Preuve

```
=== Identité : ingestion-user ===
Écrire dans RAW          -> AUTORISÉ   OK
Lire dans RAW            -> AUTORISÉ   OK
Écrire dans PROCESSED    -> REFUSÉ     OK   motif : AccessDenied
Écrire dans CURATED      -> REFUSÉ     OK   motif : AccessDenied
Supprimer dans RAW       -> REFUSÉ     OK   motif : AccessDenied
=== Identité : pipeline-user ===
Écrire dans RAW          -> REFUSÉ     OK   motif : AccessDenied
Lire dans RAW            -> AUTORISÉ   OK
Écrire dans PROCESSED    -> AUTORISÉ   OK
Écrire dans CURATED      -> AUTORISÉ   OK
Supprimer dans RAW       -> REFUSÉ     OK
=== Credentials inadaptés : ingestion-user écrit dans CURATED -> REFUSÉ (AccessDenied)
=== Bonne Access Key, mauvaise Secret Key -> REFUSÉ (SignatureDoesNotMatch)
=== Access Key inexistante -> REFUSÉ (InvalidAccessKeyId)
Bilan : 10/10 tests conformes à la matrice des droits.
```

Extrait du terminal (sortie reproduite).

Observation

Les 10 tests sont conformes. Lors d'un premier lancement, deux tests d'ingestion-user échouaient avec `SignatureDoesNotMatch` (8/10) : la Secret Key avait été mal recopiée. Une nouvelle clé a été générée et l'ancienne supprimée.

Interprétation

Deux mécanismes distincts interviennent. L'**authentification** (qui es-tu ?) rejette une Access Key inconnue (`InvalidAccessKeyId`) ou une signature invalide (`SignatureDoesNotMatch`). L'**autorisation** (as-tu le droit ?) rejette une identité valide qui sort de ses politiques (`AccessDenied`). Les responsabilités ingestion / traitement sont donc effectivement séparées par MinIO, et non par le code.

3.3 Mission 4 — Mécanisme d'ingestion

Le programme `src/ingestion/ingest.py` parcourt `incoming//`, contrôle chaque fichier (extension attendue, fichier non vide, format lisible, champs obligatoires présents) puis le dépose avec `ingestion-user` dans `raw//ingest_date=AAAA-MM-JJ/`. Les extensions et préfixes sont paramétrés dans `config/settings.json`. Le sous-préfixe daté conserve l'historique des arrivées sans écraser les lots précédents. Un fichier traité est rangé dans `incoming/_archive/`.

Test — Ingestion de plusieurs fichiers et formats

Objectif

Vérifier que les quatre sources, dans quatre formats différents, arrivent dans le bon préfixe RAW sans aucun dépôt manuel dans MinIO.

Action

Copie de `ventes.csv`, `clients.json`, `produits.xml` et `magasins.xlsx` dans `incoming/`, puis `python src/ingestion/ingest.py` et vérification avec `mc ls`.

Preuve

```
[(.venv) rekiknour@KHALESSI tp2 % cp -R ~/Desktop/mission4/src ~/Desktop/mission4/config .
[(.venv) rekiknour@KHALESSI tp2 % ls src/ingestion
python src/ingestion/ingest.py
ingest.py test_identites.py
=== Ingestion incoming/ -> s3://tp2/raw/ (2026-10-01 10:00:27) ===
4 fichier(s) détecté(s).
  2026-10-01 10:00:27 | ventes | ventes.csv | raw/ventes/ingest_date=2026-10-01/ventes.csv | 2887 o | SUCCESS | 81 enregistrement(s)
  2026-10-01 10:00:27 | clients | clients.json | raw/clients/ingest_date=2026-10-01/clients.json | 1226 o | SUCCESS | 11 enregistrement(s)
  2026-10-01 10:00:27 | produits | produits.xml | raw/produits/ingest_date=2026-10-01/produits.xml | 1596 o | SUCCESS | 10 enregistrement(s)
  2026-10-01 10:00:27 | magasins | magasins.xlsx | raw/magasins/ingest_date=2026-10-01/magasins.xlsx | 3855 o | SUCCESS | 6 enregistrement(s)

Bilan : 4 ingéré(s), 0 doublon(s) ignoré(s), 0 rejeté(s), 0 en échec (à reprendre).
Journal : /Users/rekiknour/Desktop/datalake/tp2/logs/ingestion.log
(.venv) rekiknour@KHALESSI tp2 % mc ls --recursive myaistor/tp2/raw/ | grep ingest_date
ls incoming/*
[2026-10-01 10:00:27 CEST] 1.2KiB STANDARD clients/ingest_date=2026-10-01/clients.json
[2026-10-01 10:00:27 CEST] 3.8KiB STANDARD magasins/ingest_date=2026-10-01/magasins.xlsx
[2026-10-01 10:00:27 CEST] 1.6KiB STANDARD produits/ingest_date=2026-10-01/produits.xml
[2026-10-01 10:00:27 CEST] 2.8KiB STANDARD ventes/ingest_date=2026-10-01/ventes.csv
incoming/_archive:
clients          magasins        produits        ventes

incoming/clients:

incoming/magasins:

incoming/produits:

incoming/ventes:
(.venv) rekiknour@KHALESSI tp2 % █
```

Figure 3 — Ingestion automatique des quatre sources

Observation

4 fichiers détectés, 4 SUCCESS (81, 11, 10 et 6 enregistrements). Les objets apparaissent dans raw//ingest_date=2026-10-01/ et les dossiers d'incoming/ sont vidés.

Interprétation

L'alimentation de RAW est désormais automatique, multi-fichiers et multi-formats. La limite n°1 de la V1 (dépôt manuel) est levée, de même que la limite n°2 (usage du compte administrateur).

3.4 Mission 5 — Éviter les réingestions non maîtrisées

Stratégie retenue : empreinte SHA-256 du contenu + manifest. Pour chaque fichier, le programme calcule le hash de son contenu et le compare au manifest raw/_ingestion/manifest.json, stocké dans MiniIO. Si le hash est connu, le fichier est classé DUPLICATE et n'est pas redéposé.

Critère	Pourquoi il ne suffit pas seul
Nom	Un même contenu peut être renommé ; un nouveau contenu peut porter un ancien nom
Taille	Deux fichiers différents peuvent avoir la même taille
Date	Une simple copie change la date sans changer le contenu
Hash SHA-256	Ne dépend que du contenu : retenu

Le manifest est stocké dans MinIO (et non sur le poste) afin que l'historique reste valable quelle que soit la machine qui exécute l'ingestion. Il conserve aussi la destination, la taille, le nombre de lignes et la date d'ingestion.

Test — Exécutions répétées et même fichier renommé

Objectif

Vérifier qu'un même fichier ne produit pas de réingestion, même sous un autre nom.

Action

Relance de l'ingestion sans nouveau fichier, puis dépôt de ventes.csv sous le nom ventes_copie.csv, et consultation de RAW et du manifest.

Preuve

```
((.venv) rekiknour@KHALESSI tp2 % python src/ingestion/ingest.py
=== Ingestion incoming/ -> s3://tp2/raw/ (2026-10-01 10:01:53) ===
Aucun nouveau fichier dans incoming/.
(.venv) rekiknour@KHALESSI tp2 % cp data/ventes.csv incoming/ventes/ventes_copie.csv
python src/ingestion/ingest.py
=== Ingestion incoming/ -> s3://tp2/raw/ (2026-10-01 10:02:11) ===
1 fichier(s) détecté(s).
  2026-10-01 10:02:11 | ventes | ventes_copie.csv | raw/ventes/ingest_date=2026-10-01/ventes.csv | 2887 o | DUPLICATE | contenu identique déjà ingéré le
2026-10-01 10:00:27 (ventes.csv)

Bilan : 0 ingéré(s), 1 doublon(s) ignoré(s), 0 rejeté(s), 0 en échec (à reprendre).
Journal : /Users/rekiknour/Desktop/datalake/tp2/logs/ingestion.log
(.venv) rekiknour@KHALESSI tp2 % █
```

Figure 4 — Relance à vide puis fichier renommé : DUPLICATE

```
((.venv) rekiknour@KHALESSI tp2 % mc ls --recursive myaistor/tp2/raw/ | grep ingest_date
mc cat myaistor/tp2/raw/_ingestion/manifest.json
[2026-10-01 10:00:27 CEST] 1.2KiB STANDARD clients/ingest_date=2026-10-01/clients.json
[2026-10-01 10:00:27 CEST] 3.8KiB STANDARD magasins/ingest_date=2026-10-01/magasins.xlsx
[2026-10-01 10:00:27 CEST] 1.6KiB STANDARD produits/ingest_date=2026-10-01/produits.xml
[2026-10-01 10:00:27 CEST] 2.8KiB STANDARD ventes/ingest_date=2026-10-01/ventes.csv
{
  "22a66b78ea8041a19b815703b57fc32319f1267253a87360783376f5d93608a4": {
    "fichier": "ventes.csv",
    "source": "ventes",
    "destination": "raw/ventes/ingest_date=2026-10-01/ventes.csv",
    "taille": 2887,
    "lignes": 81,
    "date": "2026-10-01 10:00:27"
  },
  "54c19923c6cb1a0342e48d6cdb5425c48fbd05455f457668105403e0e81212da": {
    "fichier": "clients.json",
    "source": "clients",
    "destination": "raw/clients/ingest_date=2026-10-01/clients.json",
    "taille": 1226,
    "lignes": 11,
    "date": "2026-10-01 10:00:27"
  },
  "96d2ba6e90b166ca4e4d772e6019ea11412f98a6797638f342177ee88970eb1d": {
    "fichier": "produits.xml",
    "source": "produits",
    "destination": "raw/produits/ingest_date=2026-10-01/produits.xml",
    "taille": 1596,
    "lignes": 10,
    "date": "2026-10-01 10:00:27"
  },
  "3772d629cf5e90cda6b3d5835971153d56e000ac48bff26ff7e68ba3b8fe0fee": {
    "fichier": "magasins.xlsx",
    "source": "magasins",
    "destination": "raw/magasins/ingest_date=2026-10-01/magasins.xlsx",
    "taille": 3855,
    "lignes": 6,
    "date": "2026-10-01 10:00:27"
  }
}
(.venv) rekiknour@KHALESSI tp2 % █
```

Figure 5 — RAW inchangé et contenu du manifest

Observation

La relance affiche « Aucun nouveau fichier ». ventes_copie.csv est classé DUPLICATE avec la référence de l'ingestion d'origine (ventes.csv, 10:00:27). RAW contient toujours 4 objets ; le manifest liste 4 empreintes.

Interprétation

La détection par contenu est robuste au renommage. L'archivage des fichiers traités évite par ailleurs de les présenter à nouveau. L'ingestion est donc rejouable sans effet de bord.

3.5 Mission 6 — Traçabilité

Chaque fichier traité produit une entrée de journal (date/heure, source, fichier, destination, taille, statut, motif) dans deux formats :

- logs/ingestion.log : une ligne lisible par un humain, au format demandé par l'énoncé ;
- logs/ingestion_journal.jsonl : un objet JSON par ligne, exploitable par un programme (Spark, pandas) pour compter les échecs ou produire des statistiques.

Justification : le journal est écrit **localement**, en ajout seulement. Il doit pouvoir tracer une indisponibilité de MinIO : stocké dans MinIO, il serait justement inaccessible pendant la panne. Le mode ajout garantit qu'aucune exécution n'efface l'historique. Les quatre statuts possibles sont SUCCESS, DUPLICATE, REJECTED (problème du fichier) et FAILED (problème d'infrastructure).

```
(.venv) rekiknour@KHALESSI tp2 % cat logs/ingestion.log
2026-10-01 10:00:27 | ventes | ventes.csv | raw/ventes/ingest_date=2026-10-01/ventes.csv | 2887 o | SUCCESS | 81 enregistrement(s)
2026-10-01 10:00:27 | clients | clients.json | raw/clients/ingest_date=2026-10-01/clients.json | 1226 o | SUCCESS | 11 enregistrement(s)
2026-10-01 10:00:27 | produits | produits.xml | raw/produits/ingest_date=2026-10-01/produits.xml | 1596 o | SUCCESS | 10 enregistrement(s)
2026-10-01 10:00:27 | magasins | magasins.xlsx | raw/magasins/ingest_date=2026-10-01/magasins.xlsx | 3855 o | SUCCESS | 6 enregistrement(s)
2026-10-01 10:02:11 | ventes | ventes_copie.csv | raw/ventes/ingest_date=2026-10-01/ventes.csv | 2887 o | DUPLICATE | contenu identique déjà ingéré le 20
26-10-01 10:00:27 (ventes.csv)
(.venv) rekiknour@KHALESSI tp2 %
```

Figure 6 — Journal lisible logs/ingestion.log

```
(.venv) rekiknour@KHALESSI tp2 % cat logs/ingestion_journal.jsonl
{"date": "2026-10-01 10:00:27", "source": "ventes", "fichier": "ventes.csv", "destination": "raw/ventes/ingest_date=2026-10-01/ventes.csv", "taille_o": 2887, "statut": "SUCCESS", "motif": "81 enregistrement(s)"}
{"date": "2026-10-01 10:00:27", "source": "clients", "fichier": "clients.json", "destination": "raw/clients/ingest_date=2026-10-01/clients.json", "taille_o": 1226, "statut": "SUCCESS", "motif": "11 enregistrement(s)"}
{"date": "2026-10-01 10:00:27", "source": "produits", "fichier": "produits.xml", "destination": "raw/produits/ingest_date=2026-10-01/produits.xml", "taille_o": 1596, "statut": "SUCCESS", "motif": "10 enregistrement(s)"}
{"date": "2026-10-01 10:00:27", "source": "magasins", "fichier": "magasins.xlsx", "destination": "raw/magasins/ingest_date=2026-10-01/magasins.xlsx", "taille_o": 3855, "statut": "SUCCESS", "motif": "6 enregistrement(s)"}
{"date": "2026-10-01 10:02:11", "source": "ventes", "fichier": "ventes_copie.csv", "destination": "raw/ventes/ingest_date=2026-10-01/ventes.csv", "taille_o": 2887, "statut": "DUPLICATE", "motif": "contenu identique déjà ingéré le 2026-10-01 10:00:27 (ventes.csv)"}
(.venv) rekiknour@KHALESSI tp2 %
```

Figure 7 — Journal structuré logs/ingestion_journal.jsonl

3.6 Mission 7 — Gestion des erreurs

Stratégie de reprise. Deux familles d'erreurs sont distinguées :

Origine	Exemples	Traitement	Reprise
Le fichier	Vide, corrompu, mauvais format, champs manquants	REJECTED, déplacé dans incoming/_rejected/	Correction humaine puis nouveau dépôt
L'infrastructure	MinIO injoignable, credentials refusés	FAILED, fichier laissé dans incoming/, arrêt immédiat, code retour 2	Automatique au lancement suivant

Le programme n'échoue jamais silencieusement : chaque cas produit un message, une ligne de journal et un code retour exploitable par un ordonnanceur (0 = succès, 1 = rejets, 2 = ingestion impossible).

Test 1 — Fichier vide

Objectif

Vérifier qu'un fichier vide est détecté et isolé.

Action

Création de `clients_vide.json` vide puis ingestion.

Preuve

```
(.venv) rekiknour@KHALESSI tp2 % touch incoming/clients/clients_vide.json
python src/ingestion/ingest.py
== Ingestion incoming/ -> s3://tp2/raw/ (2026-10-01 10:06:53) ==
1 fichier(s) détecté(s).
2026-10-01 10:06:53 | clients | clients_vide.json | - | 0 o | REJECTED | fichier vide

Bilan : 0 ingéré(s), 0 doublon(s) ignoré(s), 1 rejeté(s), 0 en échec (à reprendre).
Journal : /Users/rekiknour/Desktop/datalake/tp2/logs/ingestion.log
(.venv) rekiknour@KHALESSI tp2 %
```

Figure 8 — Fichier vide rejeté

Observation

REJECTED avec le motif « fichier vide » ; fichier déplacé dans `_rejected/clients/`.

Interprétation

Un fichier vide est un défaut de la source : le relancer ne servirait à rien, il est mis de côté.

Test 2 — Fichier corrompu ou format inattendu

Objectif

Vérifier la détection de contenus illisibles ou non conformes.

Action

Dépôt d'un JSON tronqué, d'un CSV sans les colonnes attendues et d'un fichier XML dans le dossier ventes.

Preuve

```
(.venv) rekiknour@KHALESSI tp2 % echo '{"id_client": "C099", "nom": ' > incoming/clients/clients_casse.json
echo 'colonne1,colonne2' > incoming/ventes/ventes_mauvaises_colonnes.csv
cp data/produits.xml incoming/ventes/erreur_de_dossier.xml
python src/ingestion/ingest.py
=== Ingestion incoming/ -> s3://tp2/raw/ (2026-10-01 10:07:21) ===
3 fichier(s) détecté(s).
  2026-10-01 10:07:22 | ventes | erreur_de_dossier.xml | - | 1596 o | REJECTED | extension .xml inattendue pour ventes (attendu : .csv)
  2026-10-01 10:07:22 | ventes | ventes_mauvaises_colonnes.csv | - | 18 o | REJECTED | champs obligatoires absents : date_vente, id_client, id_magasin,
id_produit, id_vente, quantite
  2026-10-01 10:07:22 | clients | clients_casse.json | - | 30 o | REJECTED | fichier corrompu ou format illisible (JSONDecodeError: Expecting value: line
2 column 1 (char 30))

Bilan : 0 ingéré(s), 0 doublon(s) ignoré(s), 3 rejeté(s), 0 en échec (à reprendre).
Journal : /Users/rekiknour/Desktop/datalake/tp2/logs/ingestion.log
(.venv) rekiknour@KHALESSI tp2 %
```

Figure 9 — Trois fichiers non conformes rejetés

Observation

Trois REJECTED avec trois motifs précis : JSONDecodeError, champs obligatoires absents, extension .xml inattendue pour ventes.

Interprétation

Les contrôles s'appliquent avant tout dépôt : RAW ne reçoit que des fichiers lisibles et structurés.

Test 3 — Fichier déjà ingéré

Objectif

Vérifier qu'un fichier connu n'est pas redéposé.

Action

Redépôt de magasins.xlsx.

Preuve

```
(.venv) rekiknour@KHALESSI tp2 % cp data/magasins.xlsx incoming/magasins/
python src/ingestion/ingest.py
=== Ingestion incoming/ -> s3://tp2/raw/ (2026-10-01 10:07:57) ===
1 fichier(s) détecté(s).
  2026-10-01 10:07:57 | magasins | magasins.xlsx | raw/magasins/ingest_date=2026-10-01/magasins.xlsx | 3855 o | DUPLICATE | contenu identique déjà ingéré
le 2026-10-01 10:00:27 (magasins.xlsx)

Bilan : 0 ingéré(s), 1 doublon(s) ignoré(s), 0 rejeté(s), 0 en échec (à reprendre).
Journal : /Users/rekiknour/Desktop/datalake/tp2/logs/ingestion.log
(.venv) rekiknour@KHALESSI tp2 %
```

Figure 10 — Fichier déjà ingéré

Observation

DUPLICATE, avec la date de la première ingestion ; RAW inchangé.

Interprétation

Le manifest joue son rôle (mission 5).

Test 4 — MinIO temporairement inaccessible

Objectif

Vérifier qu'une panne de stockage ne fait perdre aucun fichier et qu'une reprise est possible.

Action

Nouveau fichier valide ventes_0110.csv ; l'URL MinIO est remplacée par un port fermé (9999) dans settings.json, ingestion, puis rétablissement de l'URL et nouvelle ingestion.

Preuve

```
(.venv) rekiknou@KHALESSI tp2 % head -1 data/ventes.csv > incoming/ventes/ventes_0110.csv
echo "V101,2026-10-01,C001,P003,M001,2.0" >> incoming/ventes/ventes_0110.csv
echo "V102,2026-10-01,C004,P009,M002,1.0" >> incoming/ventes/ventes_0110.csv
sed -i 's#127.0.0.1:9000#127.0.0.1:9999#' config/settings.json
python src/ingestion/ingest.py
=== Ingestion incoming/ -> s3://tp2/raw/ (2026-10-01 10:08:24) ===
1 fichier(s) détecté(s).
2026-10-01 10:08:25 | ventes | ventes_0110.csv | - | 134 o | FAILED | MinIO injoignable (serveur arrêté ou réseau indisponible) – fichier conservé pou
r reprise

!! Ingestion impossible : MinIO injoignable (serveur arrêté ou réseau indisponible)
(.venv) rekiknou@KHALESSI tp2 %
```

Figure 11 — Ingestion pendant la panne : FAILED

```
(.venv) rekiknou@KHALESSI tp2 % sed -i 's#127.0.0.1:9999#127.0.0.1:9000#' config/settings.json
python src/ingestion/ingest.py
=== Ingestion incoming/ -> s3://tp2/raw/ (2026-10-01 10:09:12) ===
1 fichier(s) détecté(s).
2026-10-01 10:09:12 | ventes | ventes_0110.csv | raw/ventes/ingest_date=2026-10-01/ventes_0110.csv | 134 o | SUCCESS | 2 enregistrement(s)

Bilan : 1 ingéré(s), 0 doublon(s) ignoré(s), 0 rejeté(s), 0 en échec (à reprendre).
Journal : /Users/rekiknou/Desktop/datalake/tp2/logs/ingestion.log
(.venv) rekiknou@KHALESSI tp2 %
```

Figure 12 — Reprise après rétablissement : SUCCESS

Observation

Pendant la panne : FAILED « MinIO injoignable ... fichier conservé pour reprise », le fichier reste dans incoming/. Après rétablissement : SUCCESS, 2 enregistrements.

Interprétation

La reprise est automatique : aucune intervention sur le fichier n'est nécessaire.

Test 5 — Credentials incorrects

Objectif

Vérifier le comportement face à une Secret Key invalide.

Action

Sauvegarde du fichier de credentials, remplacement de la Secret Key, ingestion, puis restauration.

Preuve

```
(.venv) rekiknou@KHALESSI tp2 % cp config/credentials_ingestion.json config/credentials_ingestion.bak
python -c "import json;pm='config/credentials_ingestion.json';c=json.load(open(pm));c['secretKey']='MAUVAISE_CLE';json.dump(c,open(p,'w'))"
cp data/clients.json incoming/clients/clients_test_cred.json
python src/ingestion/ingest.py
=== Ingestion incoming/ -> s3://tp2/raw/ (2026-10-01 10:09:38) ===
1 fichier(s) détecté(s).
2026-10-01 10:09:38 | clients | clients_test_cred.json | - | 1226 o | FAILED | accès refusé par MinIO (SignatureDoesNotMatch) : credentials incorrects ou droits insuffisants – fichier conservé pour r
eprise

!! Ingestion impossible : accès refusé par MinIO (SignatureDoesNotMatch) : credentials incorrects ou droits insuffisants
(.venv) rekiknou@KHALESSI tp2 %
```

Figure 13 — Credentials incorrects : FAILED

Observation

FAILED « accès refusé par MinIO (SignatureDoesNotMatch) ... fichier conservé pour reprise ». Après restauration, le fichier est traité (DUPLICATE, car son contenu était déjà connu).

Interprétation

Une erreur d'accès est traitée comme une panne d'infrastructure : le fichier n'est pas en cause et reste en attente.

```
(.venv) rekiknour@KHALESSI tp2 % cat logs/ingestion.log
ls -R incoming/_rejected
2026-10-01 10:00:27 | ventes | ventes.csv | raw/ventes/ingest_date=2026-10-01/ventes.csv | 2887 o | SUCCESS | 81 enregistrement(s)
2026-10-01 10:00:27 | clients | clients.json | raw/clients/ingest_date=2026-10-01/clients.json | 1226 o | SUCCESS | 11 enregistrement(s)
2026-10-01 10:00:27 | produits | produits.xml | raw/produits/ingest_date=2026-10-01/produits.xml | 1596 o | SUCCESS | 10 enregistrement(s)
2026-10-01 10:00:27 | magasins | magasins.xlsx | raw/magasins/ingest_date=2026-10-01/magasins.xlsx | 3855 o | SUCCESS | 6 enregistrement(s)
2026-10-01 10:02:11 | ventes | ventes_copie.csv | raw/ventes/ingest_date=2026-10-01/ventes.csv | 2887 o | DUPLICATE | contenu identique déjà ingéré le 20
26-10-01 10:00:27 (ventes.csv)
2026-10-01 10:06:53 | clients | clients_vide.json | - | 0 o | REJECTED | fichier vide
2026-10-01 10:07:22 | ventes | erreur_de_dossier.xml | - | 1596 o | REJECTED | extension .xml inattendue pour ventes (attendu : .csv)
2026-10-01 10:07:25 | ventes | ventes_mauvaises_colonnes.csv | - | 18 o | REJECTED | champs obligatoires absents : date_vente, id_client, id_magasin, id_
produit, id_vente, quantite
2026-10-01 10:07:22 | clients | clients_casse.json | - | 30 o | REJECTED | fichier corrompu ou format illisible (JSONDecodeError: Expecting value: line 2
column 1 (char 30))
2026-10-01 10:07:57 | magasins | magasins.xlsx | raw/magasins/ingest_date=2026-10-01/magasins.xlsx | 3855 o | DUPLICATE | contenu identique déjà ingéré le
2026-10-01 10:00:27 (magasins.xlsx)
2026-10-01 10:08:25 | ventes | ventes_0110.csv | - | 134 o | FAILED | MinIO injoignable (serveur arrêté ou réseau indisponible) – fichier conservé pour r
eprise
2026-10-01 10:09:12 | ventes | ventes_0110.csv | raw/ventes/ingest_date=2026-10-01/ventes_0110.csv | 134 o | SUCCESS | 2 enregistrement(s)
2026-10-01 10:09:38 | clients | clients_test_cred.json | - | 1226 o | FAILED | accès refusé par MinIO (SignatureDoesNotMatch) : credentials incorrects ou
droits insuffisants – fichier conservé pour reprise
2026-10-01 10:10:18 | clients | clients_test_cred.json | raw/clients/ingest_date=2026-10-01/clients.json | 1226 o | DUPLICATE | contenu identique déjà ing
éré le 2026-10-01 10:00:27 (clients.json)
clients ventes

incoming/_rejected/clients:
clients_casse.json      clients_vide.json

incoming/_rejected/ventes:
erreur_de_dossier.xml   ventes_mauvaises_colonnes.csv
(.venv) rekiknour@KHALESSI tp2 %
```

Figure 14 — Bilan du journal : l'historique complet des tests

Commentaire. Le journal raconte toute la séquence : 4 SUCCESS initiaux, les DUPLICATE, les 4 REJECTED avec leur motif, puis les FAILED suivis de leur reprise. Les fichiers rejetés sont conservés dans incoming/_rejected/ pour analyse.

3.7 Mission 8 — Reconnecter le pipeline existant

Les traitements de la phase 1 sont réutilisés : règles de nettoyage, enrichissement, agrégations et écriture Parquet sans suppression sont inchangés. Trois adaptations seulement :

- connexion avec pipeline-user au lieu d'une identité unique ;
- lecture de **tous** les fichiers d'une source dans RAW (dépôt initial et lots ingest_date=.../), avec mémorisation du fichier et de la date d'origine de chaque ligne ;
- dédoublonnage entre lots : pour chaque identifiant, la version **la plus récemment ingérée** l'emporte, ce qui permet à une correction envoyée plus tard de remplacer l'ancienne valeur.

src/run_batch.py enchaîne ingestion puis pipeline. Si l'ingestion est impossible (code 2), le pipeline n'est pas lancé.

Test — Le pipeline lit les lots ingérés automatiquement

Objectif

Vérifier que des données déposées uniquement dans incoming/ arrivent jusqu'à CURATED.

Action

Exécution de python src/run_batch.py après l'ingestion de ventes_0110.csv (ventes V101 et V102).

Preuve

```
/Users/rekiknour/Desktop/datalake/tp2/.venv/lib/python3.13/site-packages/pyspark/sql/pandas/conversion.py:659: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
require_minimum_pandas_version()
/Users/rekiknour/Desktop/datalake/tp2/.venv/lib/python3.13/site-packages/pyspark/sql/pandas/conversion.py:936: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
require_minimum_pandas_version()
ventes : 164 ligne(s) brutes issues de 3 fichier(s)
clients : 22 ligne(s) brutes issues de 2 fichier(s)
produits : 20 ligne(s) brutes issues de 2 fichier(s)
magasins : 12 ligne(s) brutes issues de 2 fichier(s)

[2/4] RAW -> PROCESSED (nettoyage + Parquet)
ventes : 82 ligne(s) après nettoyage
-> s3a://tp2/processed/ventes/ (1 fichier(s) part + _SUCCESS)
clients : 10 ligne(s) après nettoyage
-> s3a://tp2/processed/clients/ (1 fichier(s) part + _SUCCESS)
produits : 10 ligne(s) après nettoyage
-> s3a://tp2/processed/produits/ (1 fichier(s) part + _SUCCESS)
magasins : 6 ligne(s) après nettoyage
-> s3a://tp2/processed/magasins/ (1 fichier(s) part + _SUCCESS)

[3/4] Enrichissement des ventes (lecture depuis PROCESSED)
ventes exploitables : 78 | écarts : 4
+-----+-----+-----+-----+-----+
|id_vente|id_produit|quantite|prix_unitaire|anomalies|
+-----+-----+-----+-----+-----+
|V012|P010|-2|21.5|quantite negative|
|V053|P004|0|59.0|quantite nulle|
|V032|P999|2|NULL|produit inconnu|
|V018|P003|NULL|49.9|quantite absente|
+-----+-----+-----+-----+-----+
-> s3a://tp2/processed/ventes_enrichies/ (1 fichier(s) part + _SUCCESS)

[4/4] PROCESSED -> CURATED (agregations)
-> s3a://tp2/curated/ca_par_ville/ (1 fichier(s) part + _SUCCESS)
-> s3a://tp2/curated/ca_par_categorie/ (1 fichier(s) part + _SUCCESS)

=== Contrôle des résultats CURATED (reus depuis MinIO) ===
+-----+-----+-----+-----+
|ville|nb_ventes|quantite_totale|chiffre_affaires|
+-----+-----+-----+-----+
|Lille|15|62|2093.4|
|Paris|17|55|2027.5|
|Lyon|12|49|1976.7|
|Rennes|12|38|1354.25|
|Marseille|13|42|1350.2|
|Nantes|8|25|1049.6|
|Inconnue|1|5|199.5|
+-----+-----+-----+-----+

+-----+-----+-----+-----+
|categorie|nb_ventes|quantite_totale|chiffre_affaires|
+-----+-----+-----+-----+
|Audio|19|63|3544.1|
|Informatique|30|107|3525.7|
|Accessoires|13|53|1940.9|
|Maison|10|34|631.95|
|Non renseignée|6|19|408.5|
+-----+-----+-----+-----+

Pipeline terminé avec succès. (raw/images_produits/ conservées dans RAW uniquement)
(.venv) rekiknour@KHALESSI tp2 %
```

Figure 15 — Pipeline PySpark existant relancé sur RAW

Observation

Les ventes sont lues depuis 3 fichiers ; 78 ventes exploitables au lieu de 76. Paris passe de 1 982,60 à 2 027,50 (V102) et Lyon de 1 876,90 à 1 976,70 (V101).

Interprétation

La chaîne incoming → ingestion → RAW → PySpark → PROCESSED → CURATED fonctionne sans intervention dans MinIO, en réutilisant le pipeline existant.

3.8 Mission 9 — Tester l'ingestion avec un nouveau lot

Un lot différent de celui du développement (lot_test_0210) a été constitué :

Fichier	Cas testé	Attendu
ventes_0210.csv	Fichier valide : 8 nouvelles ventes (dont V208 en quantité 0)	SUCCESS
clients_0210.json	Mise à jour : nouvelle cliente C011, ville de C007 renseignée	SUCCESS
produits_0210.xml	Mise à jour : nouveau produit P011, catégorie de P010 renseignée	SUCCESS
magasins_renvoi.xlsx	Fichier déjà ingéré (copie de magasins.xlsx)	DUPLICATE
ventes_vide.csv	Fichier vide	REJECTED
clients_incomplet.json	Champ segment absent	REJECTED

Test — Lot de six fichiers arrivant ensemble

Objectif

Vérifier la robustesse de l'ingestion sur des données nouvelles, sans intervention manuelle dans MinIO.

Action

Dépôt simultané des six fichiers dans incoming/, puis python src/run_batch.py.

Preuve

```
(.venv) rekiknou@KHALESSI tp2 % python src/run_batch.py
##### ÉTAPE 1/2 - INGESTION (ingestion-user) #####
=== Ingestion incoming/ -> s3://tp2/raw/ (2026-10-01 10:21:06) ===
6 fichier(s) détecté(s).
2026-10-01 10:21:06 | ventes | ventes_0210.csv | raw/ventes/ingest_date=2026-10-01/ventes_0210.csv | 341 o | SUCCESS | 8 enregistrement(s)
2026-10-01 10:21:06 | ventes | ventes_vide.csv | - | 0 o | REJECTED | fichier vide
2026-10-01 10:21:06 | clients | clients_0210.json | raw/clients/ingest_date=2026-10-01/clients_0210.json | 191 o | SUCCESS | 2 enregistrement(s)
2026-10-01 10:21:06 | clients | clients_incomplet.json | - | 69 o | REJECTED | champs obligatoires absents : segment
2026-10-01 10:21:06 | produits | produits_0210.xml | raw/produits/ingest_date=2026-10-01/produits_0210.xml | 372 o | SUCCESS | 2 enregistrement(s)
2026-10-01 10:21:07 | magasins | magasins_renvoy.xlsx | raw/magasins/ingest_date=2026-10-01/magasins.xlsx | 3855 o | DUPLICATE | contenu identique déjà ingéré le 2026-10-01 10:00:27 (magasins.xlsx)

Bilan : 3 ingéré(s), 1 doublon(s) ignoré(s), 2 rejeté(s), 0 en échec (à reprendre).
Journal : /Users/rekiknou/Desktop/datalake/tp2/logs/ingestion.log
```

Figure 16 — Ingestion du lot de test

```
require_minimum_pandas_version()
ventes : 172 ligne(s) brutes issues de 4 fichier(s)
clients : 24 ligne(s) brutes issues de 3 fichier(s)
produits : 22 ligne(s) brutes issues de 3 fichier(s)
magasins : 12 ligne(s) brutes issues de 2 fichier(s)

[2/4] RAW -> PROCESSED (nettoyage + Parquet)
ventes : 90 ligne(s) après nettoyage
-> s3a://tp2/processed/ventes/ (1 fichier(s) part + _SUCCESS)
clients : 11 ligne(s) après nettoyage
-> s3a://tp2/processed/clients/ (1 fichier(s) part + _SUCCESS)
produits : 11 ligne(s) après nettoyage
-> s3a://tp2/processed/produits/ (1 fichier(s) part + _SUCCESS)
magasins : 6 ligne(s) après nettoyage
-> s3a://tp2/processed/magasins/ (1 fichier(s) part + _SUCCESS)

[3/4] Enrichissement des ventes (lecture depuis PROCESSED)
ventes exploitables : 85 | écartées : 5
+-----+-----+-----+-----+-----+
| id_vente | id_produit | quantite | prix_unitaire | anomalies |
+-----+-----+-----+-----+-----+
| V012 | P010 | -2 | 21.5 | quantite negative |
| V053 | P004 | 0 | 59.0 | quantite nulle |
| V208 | P007 | 0 | 39.9 | quantite nulle |
| V052 | P009 | 2 | NULL | produit inconnu |
| V018 | P003 | NULL | 49.9 | quantite absente |
+-----+-----+-----+-----+-----+

-> s3a://tp2/processed/ventes_enrichies/ (1 fichier(s) part + _SUCCESS)

[4/4] PROCESSED -> CURATED (agrégations)
-> s3a://tp2/curated/ca_par_ville/ (1 fichier(s) part + _SUCCESS)
-> s3a://tp2/curated/ca_par_categorie/ (1 fichier(s) part + _SUCCESS)

=== Contrôle des résultats CURATED (re lus depuis MinIO) ===
+-----+-----+-----+-----+
| ville | nb_ventes | quantite_totale | chiffre_affaires |
+-----+-----+-----+-----+
| Paris | 18 | 60 | 2150.0 |
| Lille | 16 | 64 | 2136.4 |
| Lyon | 13 | 52 | 2066.4 |
| Rennes | 13 | 42 | 1421.85 |
| Marseille | 14 | 43 | 1409.2 |
| Nantes | 10 | 28 | 1286.6 |
| Inconnue | 1 | 5 | 199.5 |
+-----+-----+-----+-----+

+-----+-----+-----+-----+
| categorie | nb_ventes | quantite_totale | chiffre_affaires |
+-----+-----+-----+-----+
| Audio | 22 | 67 | 3840.1 |
| Informatique | 32 | 115 | 3737.9 |
| Accessoires | 20 | 74 | 2392.4 |
| Maison | 11 | 38 | 699.55 |
+-----+-----+-----+-----+

Pipeline terminé avec succès. (raw/images_produits/ conservées dans RAW uniquement)
(.venv) rekiknou@KHALESSI tp2 %
```

Figure 17 — Résultats PROCESSED et CURATED

Observation

Bilan : 3 ingérés, 1 doublon, 2 rejetés. Le pipeline lit 4 fichiers de ventes (90 ventes, 85 exploitables, 5 écartées dont V208). Paris atteint 2 150,00 et la catégorie « Non renseignée » disparaît.

Interprétation

Les règles de qualité de la phase 1 s'appliquent aux nouvelles données. La disparition de « Non renseignée » prouve que la correction de P010 a remplacé l'ancienne valeur grâce à la règle de la version la plus récente.

4. Architecture V2 — Ingestion automatisée

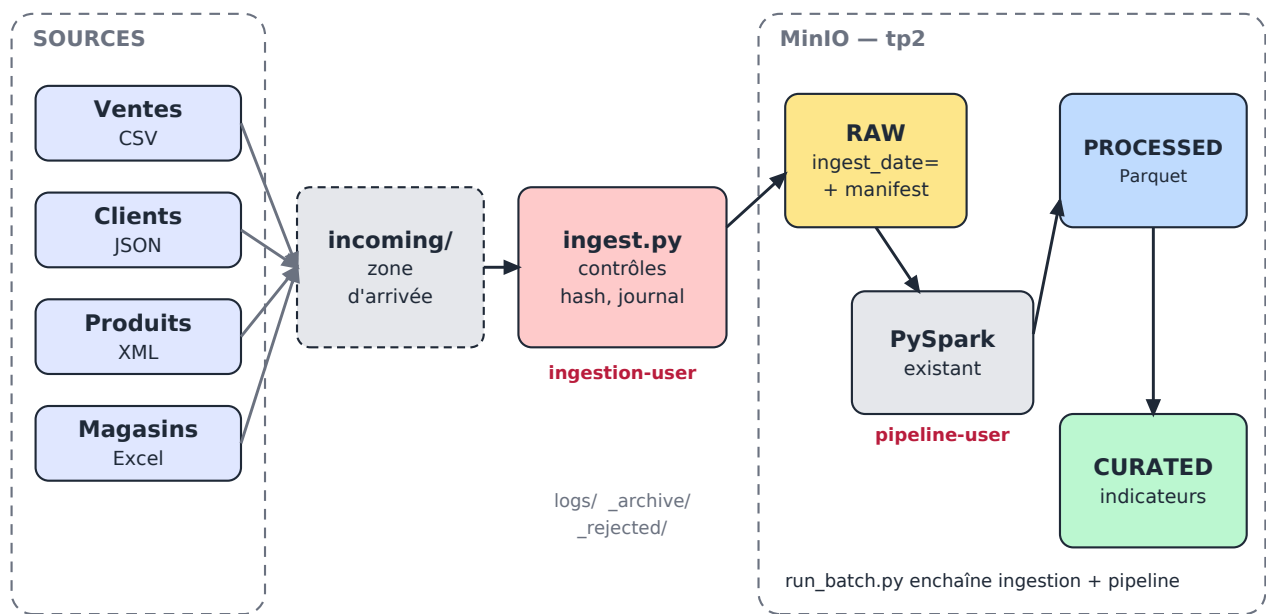


Figure 18 — Architecture V2 — Ingestion automatisée

Aspect	V1	V2
Arrivée des données	Dépôt manuel mc cp	Dépôt dans incoming/, ingestion automatique
Identité qui écrit dans RAW	minioadmin (tous les droits)	ingestion-user (RAW uniquement)
Contrôles à l'arrivée	Aucun	Extension, vide, format, champs obligatoires
Doublons	Non détectés	Hash SHA-256 + manifest
Traçabilité	Aucune	Journal .log et .jsonl, archives, rejets
Erreurs	Non gérées	REJECTED / FAILED, reprise automatique
Organisation de RAW	Un fichier par source	Historique par ingest_date
Pipeline	Lancé à la main, lit un fichier	Réutilisé, lit tous les lots, enchaîné par run_batch.py

5. Phase 3 — Intégrer une ingestion streaming

5.1 Mission 10 — Architecture cible (conçue avant le code)

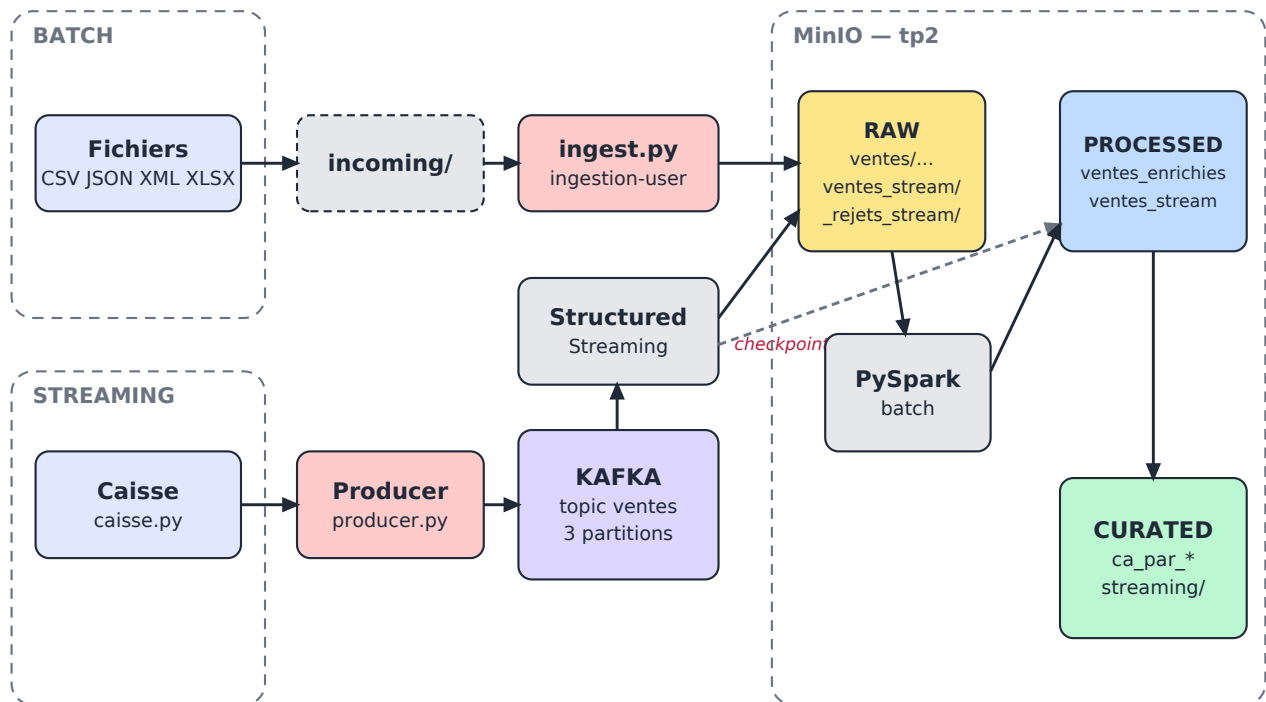


Figure 19 — Architecture cible — batch et streaming

Composant	Rôle	Flux
Simulateur de caisse	Génère des événements de vente autonomes à un rythme configurable	→ Producer
Producer Kafka	Publie chaque vente (JSON) dans le topic ventes, clé = magasin	→ Kafka
Kafka (broker)	Stocke durablement les événements, dans l'ordre par partition ; découple producteurs et consommateurs ; sert de tampon en cas d'arrêt du traitement	→ Spark
Spark Structured Streaming	Lit Kafka par micro-batches, contrôle le schéma, alimente RAW, PROCESSED, CURATED ; mémorise sa position dans un checkpoint	→ MinIO
MinIO (RAW / PROCESSED / CURATED)	Stockage commun au batch et au streaming, préfixes séparés	—
Ingestion batch + PySpark	Chaîne de la phase 2, inchangée	→ MinIO

5.2 Mission 11 — Kafka en local

Kafka 4.3.1 est installé avec Homebrew et démarré en mode **KRaft** (sans ZooKeeper). Le topic ventes est créé avec 3 partitions et un facteur de réplication de 1 (un seul broker).

Test — Fonctionnement du topic

Objectif

Vérifier que le topic existe et qu'un message écrit peut être relu.

Action

kafka-topics --describe, envoi de trois messages texte avec kafka-console-producer, relecture avec kafka-console-consumer --from-beginning.

Preuve

```
Topic: ventes PartitionCount: 3 ReplicationFactor: 1
  Partition: 0 Leader: 1 Replicas: 1 Isr: 1
  Partition: 1 Leader: 1 Replicas: 1 Isr: 1
  Partition: 2 Leader: 1 Replicas: 1 Isr: 1
$ kafka-console-consumer --topic ventes --from-beginning --timeout-ms 5000
bonjour kafka
test vente 1
test vente 2
Processed a total of 3 messages
```

Extrait du terminal (sortie reproduite).

Observation

Le topic compte 3 partitions, toutes gérées par le broker 1. Les trois messages sont relus. Un premier essai avait échoué (0 message) car les messages avaient été tapés sur une seule ligne sans valider : le producer console n'envoie un message qu'à l'appui sur Entrée.

Interprétation

Le broker est opérationnel. Le TimeoutException affiché à la fin est normal : le consumer s'arrête après 5 s sans nouveau message, comme demandé.

5.3 Missions 12 et 13 — Simulateur de caisse et Producer

caisse.py génère des ventes **autonomes** : chaque événement contient identifiant, horodatage, caisse, magasin, ville, client, produit, catégorie, quantité, prix et version de schéma. Le rythme (--rythme) et le nombre (--nombre) sont configurables ; --taux-invalides injecte des événements hors schéma. producer.py publie chaque vente avec acks=all (confirmation du broker) et la **clé id_magasin**, afin que toutes les ventes d'un magasin aillent dans la même partition et conservent leur ordre.

```
[notice] io update, run: pip install --upgrade pip
[.venv] rekiknour@KHALESSI tp2 % python src/streaming/caisse.py --rythme 1 --nombre 5
{"schema_version": 1, "id_vente": "S-B7641010F9", "horodatage": "2026-10-01T10:34:20.763", "id_caisse": "M006-K1", "id_magasin": "M006", "ville": "Lille", "id_client": "C001", "id_produit": "P003", "nom_produit": "Casque audio", "categorie": "Audio", "quantite": 2, "prix_unitaire": 49.9}
{"schema_version": 1, "id_vente": "S-59666E2598", "horodatage": "2026-10-01T10:34:21.768", "id_caisse": "M003-K3", "id_magasin": "M003", "ville": "Nantes", "id_client": "C002", "id_produit": "P001", "nom_produit": "Clavier sans fil", "categorie": "Informatique", "quantite": 3, "prix_unitaire": 29.9}
{"schema_version": 1, "id_vente": "S-1A4AFC467D", "horodatage": "2026-10-01T10:34:22.773", "id_caisse": "M002-K1", "id_magasin": "M002", "ville": "Paris", "id_client": "C008", "id_produit": "P007", "nom_produit": "Sac à dos", "categorie": "Accessoires", "quantite": 1, "prix_unitaire": 39.9}
{"schema_version": 1, "id_vente": "S-10898828C8", "horodatage": "2026-10-01T10:34:23.779", "id_caisse": "M002-K3", "id_magasin": "M002", "ville": "Paris", "id_client": "C006", "id_produit": "P003", "nom_produit": "Casque audio", "categorie": "Audio", "quantite": 2, "prix_unitaire": 49.9}
{"schema_version": 1, "id_vente": "S-6A3F9F5604", "horodatage": "2026-10-01T10:34:24.784", "id_caisse": "M005-K3", "id_magasin": "M005", "ville": "Rennes", "id_client": "C011", "id_produit": "P007", "nom_produit": "Sac à dos", "categorie": "Accessoires", "quantite": 1, "prix_unitaire": 39.9}
[.venv] rekiknour@KHALESSI tp2 %
```

Figure 20 — Mission 12 — Cinq ventes générées à une seconde d'intervalle

Test — Publication dans le topic ventes

Objectif

Vérifier que Kafka reçoit effectivement les événements du simulateur.

Action

producer.py --rythme 1 --nombre 10, puis relecture du topic depuis le début.

Preuve

```
[(.venv) rekiknour@KHALESSI tp2 % python src/streaming/producer.py --rythme 1 --nombre 10 ]
Envoi vers le topic 'ventes' (rythme 1.0s, Ctrl+C pour arrêter)
#1 S-37980588FC Paris Accessoires x5 -> partition 1, offset 0
#2 S-1684B04B6F Marseille Accessoires x2 -> partition 1, offset 1
#3 S-5AA7959855 Paris Accessoires x1 -> partition 1, offset 2
#4 S-69B27B1159 Rennes Audio x2 -> partition 1, offset 3
#5 S-5E81579B36 Lyon Informatique x3 -> partition 1, offset 4
#6 S-3E1589C6C9 Marseille Accessoires x1 -> partition 1, offset 5
#7 S-4A46AC35EE Paris Accessoires x1 -> partition 1, offset 6
#8 S-AF1B954BD8 Rennes Accessoires x1 -> partition 1, offset 7
#9 S-2B783F9D9B Nantes Informatique x4 -> partition 0, offset 0
#10 S-6392974329 Paris Informatique x2 -> partition 1, offset 8

Producer arrêté : 10 message(s) envoyé(s), 0 échec(s).
[(.venv) rekiknour@KHALESSI tp2 % kafka-console-consumer --bootstrap-server localhost:9092 --topic ventes --from-
m-beginning --timeout-ms 5000
OpenJDK 64-Bit Server VM warning: Option InitiatingHeapOccupancyPercent was deprecated in version 27.0 and will
l likely be removed in a future release. Use option G1IHOP instead.
The consumer rebalance protocol (KIP-848) is production-ready! Set group.protocol=consumer to try it out. See
https://kafka.apache.org/documentation/#consumer_rebalance_protocol
{"schema_version": 1, "id_vente": "S-2B783F9D9B", "horodatage": "2026-10-01T10:35:15.619", "id_caisse": "M003-
K3", "id_magasin": "M003", "ville": "Nantes", "id_client": "C010", "id_produit": "P001", "nom_produit": "Clavi
er sans fil", "categorie": "Informatique", "quantite": 4, "prix_unitaire": 29.9}
bonjour kafka
test vente 1
test vente 2
{"schema_version": 1, "id_vente": "S-37980588FC", "horodatage": "2026-10-01T10:35:07.356", "id_caisse": "M002-
K1", "id_magasin": "M002", "ville": "Paris", "id_client": "C006", "id_produit": "P010", "nom_produit": "Charge
ur USB-C", "categorie": "Accessoires", "quantite": 5, "prix_unitaire": 21.5}
{"schema_version": 1, "id_vente": "S-1684B04B6F", "horodatage": "2026-10-01T10:35:08.478", "id_caisse": "M004-
K2", "id_magasin": "M004", "ville": "Marseille", "id_client": "C004", "id_produit": "P010", "nom_produit": "Ch
argeur USB-C", "categorie": "Accessoires", "quantite": 2, "prix_unitaire": 21.5}
{"schema_version": 1, "id_vente": "S-5AA7959855", "horodatage": "2026-10-01T10:35:09.505", "id_caisse": "M002-
K3", "id_magasin": "M002", "ville": "Paris", "id_client": "C006", "id_produit": "P010", "nom_produit": "Charge
ur USB-C", "categorie": "Accessoires", "quantite": 1, "prix_unitaire": 21.5}
{"schema_version": 1, "id_vente": "S-69B27B1159", "horodatage": "2026-10-01T10:35:10.527", "id_caisse": "M005-
K1", "id_magasin": "M005", "ville": "Rennes", "id_client": "C004", "id_produit": "P003", "nom_produit": "Casqu
e audio", "categorie": "Audio", "quantite": 2, "prix_unitaire": 49.9}
{"schema_version": 1, "id_vente": "S-5E81579B36", "horodatage": "2026-10-01T10:35:11.546", "id_caisse": "M001-
K1", "id_magasin": "M001", "ville": "Lyon", "id_client": "C004", "id_produit": "P002", "nom_produit": "Souris
ergonomique", "categorie": "Informatique", "quantite": 3, "prix_unitaire": 24.5}
{"schema_version": 1, "id_vente": "S-3E1589C6C9", "horodatage": "2026-10-01T10:35:12.566", "id_caisse": "M004-
K2", "id_magasin": "M004", "ville": "Marseille", "id_client": "C011", "id_produit": "P008", "nom_produit": "Su
pport ordinateur", "categorie": "Accessoires", "quantite": 1, "prix_unitaire": 32.0}
{"schema_version": 1, "id_vente": "S-4A46AC35EE", "horodatage": "2026-10-01T10:35:13.581", "id_caisse": "M002-
K2", "id_magasin": "M002", "ville": "Paris", "id_client": "C007", "id_produit": "P010", "nom_produit": "Charge
ur USB-C", "categorie": "Accessoires", "quantite": 1, "prix_unitaire": 21.5}
{"schema_version": 1, "id_vente": "S-AF1B954BD8", "horodatage": "2026-10-01T10:35:14.602", "id_caisse": "M005-
K3", "id_magasin": "M005", "ville": "Rennes", "id_client": "C006", "id_produit": "P007", "nom_produit": "Sac à
dos", "categorie": "Accessoires", "quantite": 1, "prix_unitaire": 39.9}
{"schema_version": 1, "id_vente": "S-6392974329", "horodatage": "2026-10-01T10:35:16.637", "id_caisse": "M002-
K1", "id_magasin": "M002", "ville": "Paris", "id_client": "C008", "id_produit": "P002", "nom_produit": "Souris
```

Figure 21 — Producer (10 envois) et consumer (13 messages)

Observation

Les 10 envois sont confirmés avec leur partition et leur offset. Le consumer relit 13 messages (3 tests + 10 ventes), mais pas dans l'ordre d'envoi : Nantes (partition 0) apparaît en premier.

Interprétation

Les ventes d'un même magasin vont toujours dans la même partition (Nantes en p0, les autres villes en p1). Kafka garantit l'ordre **à l'intérieur** d'une partition, pas entre partitions. Chaque partition a sa propre numérotation d'offsets.

5.4 Mission 14 — Observer le fonctionnement de Kafka

Notion	Définition	Dans notre projet
Message	Donnée publiée, avec une clé facultative	Une vente en JSON, clé = id_magasin
Topic	Flux nommé de messages	ventes
Partition	Sous-ensemble ordonné d'un topic, unité de parallélisme	3 partitions
Offset	Position d'un message dans sa partition, immuable	p1:0, 1, 2...
Producer	Application qui écrit dans un topic	producer.py
Consumer	Application qui lit un topic	kafka-console-consumer, Spark
Consumer group	Groupe de consumers ; Kafka mémorise l'offset commité par groupe	groupe-test

Notion	Définition	Dans notre projet
Broker	Serveur Kafka qui stocke les partitions	Broker id 1 (KRaft)
Lag	Messages disponibles non encore lus par un groupe	Colonne LAG

Test — Consumer arrêté puis redémarré

Objectif

Observer ce qu'il advient des messages produits pendant l'arrêt d'un consumer.

Action

Consommer du groupe groupe-test démarré pendant que le producer émet une vente toutes les 2 s ; arrêter du consumer ; observation du groupe avec kafka-consumer-groups --describe ; redémarrage sans --from-beginning.

Preuve

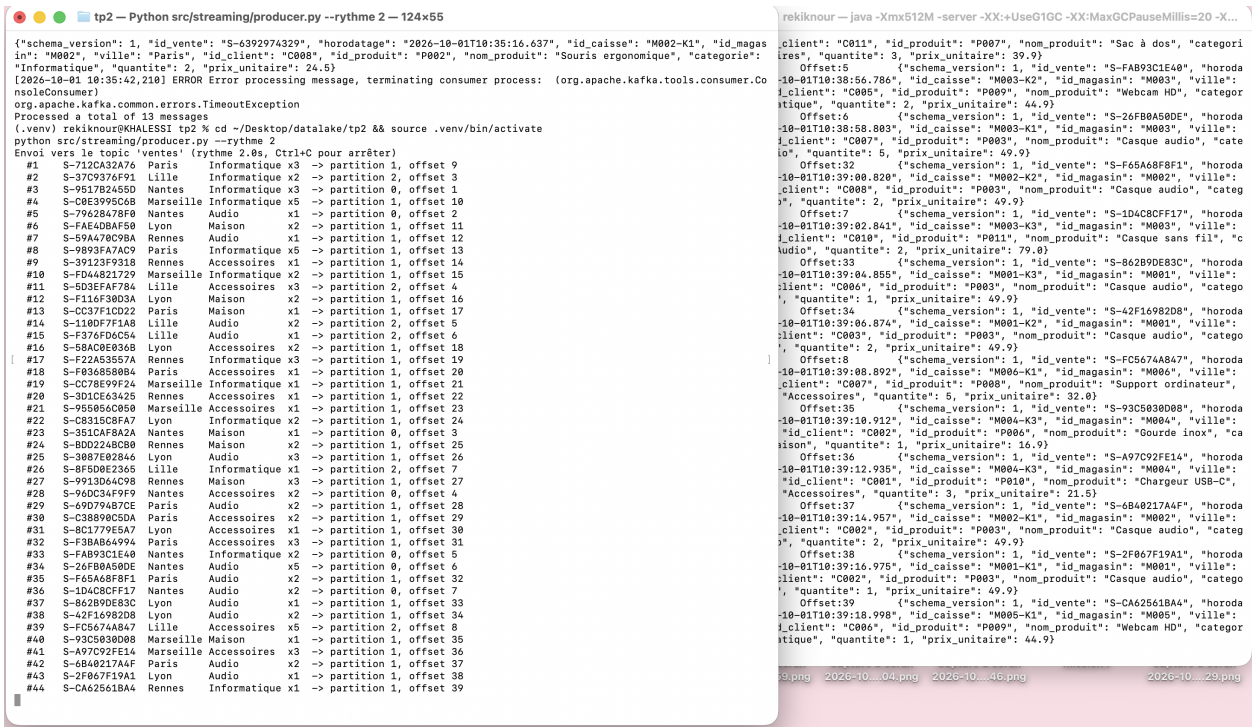


Figure 22 — A — Production et consommation en direct

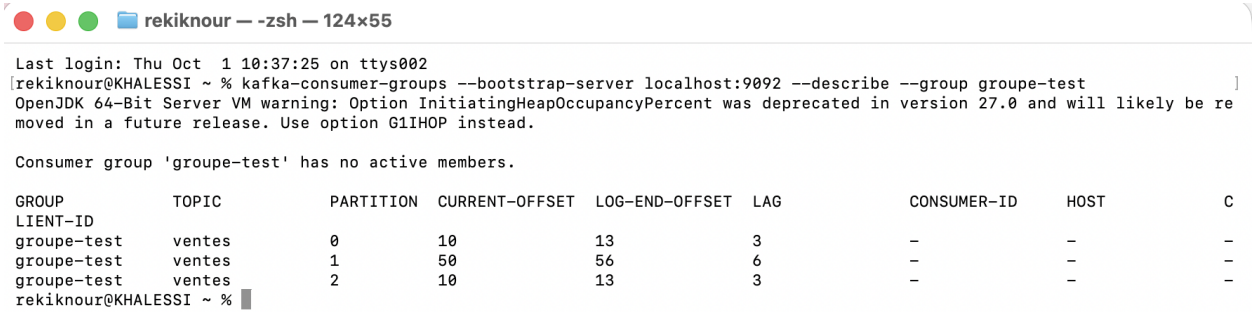


Figure 23 — B — Consumer arrêté : le LAG s'accumule

```
rekiknour — -zsh — 136x56
Partition:1 Offset:71 {"schema_version": 1, "id_vente": "S-2647BC7330", "horodatage": "2026-10-01T10:41:02.018", "id_caisse": "M005-K1", "id_magasin": "M005", "ville": "Rennes", "id_client": "C010", "id_produit": "P011", "nom_produit": "Casque sans fil", "categorie": "Audio", "quantite": 1, "prix_unitaire": 79.0}
Partition:1 Offset:72 {"schema_version": 1, "id_vente": "S-6E183A21C8", "horodatage": "2026-10-01T10:41:04.041", "id_caisse": "M005-K1", "id_magasin": "M005", "ville": "Rennes", "id_client": "C011", "id_produit": "P006", "nom_produit": "Gourde inox", "categorie": "Maison", "quantite": 3, "prix_unitaire": 16.9}
Partition:1 Offset:73 {"schema_version": 1, "id_vente": "S-602D70A1E0", "horodatage": "2026-10-01T10:41:06.059", "id_caisse": "M004-K1", "id_magasin": "M004", "ville": "Lyon", "id_client": "C001", "id_produit": "P008", "nom_produit": "Support ordinateur", "categorie": "Accessoires", "quantite": 1, "prix_unitaire": 32.0}
Partition:1 Offset:74 {"schema_version": 1, "id_vente": "S-0ADF94F489", "horodatage": "2026-10-01T10:41:08.081", "id_caisse": "M004-K1", "id_magasin": "M004", "ville": "Marseille", "id_client": "C005", "id_produit": "P011", "nom_produit": "Casque sans fil", "categorie": "Audio", "quantite": 2, "prix_unitaire": 79.0}
Partition:1 Offset:75 {"schema_version": 1, "id_vente": "S-7EF7A1F256", "horodatage": "2026-10-01T10:41:10.102", "id_caisse": "M004-K3", "id_magasin": "M004", "ville": "Marseille", "id_client": "C009", "id_produit": "P006", "nom_produit": "Gourde inox", "categorie": "Maison", "quantite": 2, "prix_unitaire": 16.9}
Partition:2 Offset:19 {"schema_version": 1, "id_vente": "S-5447CC9CD5", "horodatage": "2026-10-01T10:41:12.125", "id_caisse": "M006-K2", "id_magasin": "M006", "ville": "Lille", "id_client": "C003", "id_produit": "P008", "nom_produit": "Support ordinateur", "categorie": "Accessoires", "quantite": 3, "prix_unitaire": 32.0}
Partition:1 Offset:76 {"schema_version": 1, "id_vente": "S-09ED96A388", "horodatage": "2026-10-01T10:41:14.148", "id_caisse": "M005-K3", "id_magasin": "M005", "ville": "Rennes", "id_client": "C008", "id_produit": "P011", "nom_produit": "Casque sans fil", "categorie": "Audio", "quantite": 2, "prix_unitaire": 79.0}
Partition:1 Offset:77 {"schema_version": 1, "id_vente": "S-5ACDA9B2AF", "horodatage": "2026-10-01T10:41:16.171", "id_caisse": "M002-K1", "id_magasin": "M002", "ville": "Paris", "id_client": "C011", "id_produit": "P007", "nom_produit": "Sac à dos", "categorie": "Accessoires", "quantite": 4, "prix_unitaire": 39.9}
Partition:1 Offset:78 {"schema_version": 1, "id_vente": "S-3E89274AF6", "horodatage": "2026-10-01T10:41:18.191", "id_caisse": "M005-K3", "id_magasin": "M005", "ville": "Rennes", "id_client": "C006", "id_produit": "P006", "nom_produit": "Gourde inox", "categorie": "Maison", "quantite": 1, "prix_unitaire": 16.9}
Partition:1 Offset:79 {"schema_version": 1, "id_vente": "S-B254A7778D", "horodatage": "2026-10-01T10:41:20.211", "id_caisse": "M001-K2", "id_magasin": "M001", "ville": "Lyon", "id_client": "C005", "id_produit": "P002", "nom_produit": "Souris ergonomique", "categorie": "Informatique", "quantite": 1, "prix_unitaire": 24.5}
Partition:2 Offset:20 {"schema_version": 1, "id_vente": "S-8491908A6E", "horodatage": "2026-10-01T10:41:22.231", "id_caisse": "M006-K1", "id_magasin": "M006", "ville": "Lille", "id_client": "C003", "id_produit": "P004", "nom_produit": "Enceinte Bluetooth", "categorie": "Audio", "quantite": 2, "prix_unitaire": 59.0}
Partition:2 Offset:21 {"schema_version": 1, "id_vente": "S-85004CC09A", "horodatage": "2026-10-01T10:41:24.253", "id_caisse": "M006-K1", "id_magasin": "M006", "ville": "Lille", "id_client": "C004", "id_produit": "P005", "nom_produit": "Lampe LED", "categorie": "Maison", "quantite": 4, "prix_unitaire": 18.75}
Partition:2 Offset:22 {"schema_version": 1, "id_vente": "S-2390576FDB", "horodatage": "2026-10-01T10:41:26.274", "id_caisse": "M006-K3", "id_magasin": "M006", "ville": "Lille", "id_client": "C008", "id_produit": "P008", "nom_produit": "Support ordinateur", "categorie": "Accessoires", "quantite": 4, "prix_unitaire": 32.0}
Partition:0 Offset:17 {"schema_version": 1, "id_vente": "S-75D35FC864", "horodatage": "2026-10-01T10:41:28.293", "id_caisse": "M003-K1", "id_magasin": "M003", "ville": "Nantes", "id_client": "C007", "id_produit": "P010", "nom_produit": "Chargeur USB-C", "categorie": "Accessoires", "quantite": 3, "prix_unitaire": 21.5}
Partition:0 Offset:18 {"schema_version": 1, "id_vente": "S-812BCAD4ED", "horodatage": "2026-10-01T10:41:30.314", "id_caisse": "M003-K3", "id_magasin": "M003", "ville": "Nantes", "id_client": "C001", "id_produit": "P003", "nom_produit": "Casque audio", "categorie": "Audio", "quantite": 1, "prix_unitaire": 49.9}
Partition:0 Offset:19 {"schema_version": 1, "id_vente": "S-87B2704B66", "horodatage": "2026-10-01T10:41:32.332", "id_caisse": "M003-K2", "id_magasin": "M003", "ville": "Nantes", "id_client": "C003", "id_produit": "P010", "nom_produit": "Chargeur USB-C", "categorie": "Accessoires", "quantite": 1, "prix_unitaire": 21.5}
Partition:2 Offset:23 {"schema_version": 1, "id_vente": "S-E6C00BF377", "horodatage": "2026-10-01T10:41:34.350", "id_caisse": "M006-K2", "id_magasin": "M006", "ville": "Lille", "id_client": "C009", "id_produit": "P011", "nom_produit": "Casque sans fil", "categorie": "Audio", "quantite": 4, "prix_unitaire": 79.0}
Partition:0 Offset:24 {"schema_version": 1, "id_vente": "S-BDD5E8BFC0", "horodatage": "2026-10-01T10:41:36.370", "id_caisse": "M006-K2", "id_magasin": "M006", "ville": "Lille", "id_client": "C007", "id_produit": "P006", "nom_produit": "Gourde inox", "categorie": "Maison", "quantite": 3, "prix_unitaire": 16.9}
^CProcessed a total of 55 messages
rekiknour@KHALESSI ~ %
```

Figure 24 — C — Redémarrage : rattrapage depuis l'offset commité

Observation

Pendant l'arrêt, le groupe n'a plus de membre actif et le LAG vaut 3, 6 et 3 selon les partitions (par exemple offset commité 50 contre 56 disponibles en p1). Au redémarrage, le consumer affiche immédiatement les messages manqués puis reprend en direct.

Interprétation

Les messages ne sont pas perdus : Kafka les conserve, indépendamment de leur lecture. Le groupe reprend à son **dernier offset commité**, sans tout relire. Contrairement à une file classique, la lecture ne supprime rien : chaque groupe a sa propre position, et un autre groupe pourrait relire depuis l'offset 0.

```

rekiknour — java -Xmx1G -Xms1G -server -XX:+UseG1GC -XX:MaxGCPauseMillis=20 -XX:Initia...

[2026-10-01 10:41:04,132] INFO [GroupCoordinator id=1 topic=__consumer_offsets partition=25] Dynamic
member with unknown member id joins group groupe-test in Empty state. Created a new member id console
-consumer-0f0146ce-4b8b-45d1-abca-9c8bdd1bf6dd and requesting the member to rejoin with this id. (org
.apache.kafka.coordinator.group.GroupMetadataManager)
[2026-10-01 10:41:04,134] INFO [GroupCoordinator id=1 topic=__consumer_offsets partition=25] Pending
dynamic member with id console-consumer-0f0146ce-4b8b-45d1-abca-9c8bdd1bf6dd joins group groupe-test
in Empty state. Adding to the group now. (org.apache.kafka.coordinator.group.GroupMetadataManager)
[2026-10-01 10:41:04,135] INFO [GroupCoordinator id=1 topic=__consumer_offsets partition=25] Preparin
g to rebalance group groupe-test in state PreparingRebalance with old generation 2 (reason: Adding ne
w member console-consumer-0f0146ce-4b8b-45d1-abca-9c8bdd1bf6dd with group instance id null; client re
ason: need to re-join with the given member-id: console-consumer-0f0146ce-4b8b-45d1-abca-9c8bdd1bf6dd
). (org.apache.kafka.coordinator.group.GroupMetadataManager)
[2026-10-01 10:41:07,137] INFO [GroupCoordinator id=1 topic=__consumer_offsets partition=25] Stabiliz
ed group groupe-test generation 3 with 1 members. (org.apache.kafka.coordinator.group.GroupMetadataMa
nager)
[2026-10-01 10:41:07,160] INFO [GroupCoordinator id=1 topic=__consumer_offsets partition=25] Assignme
nt received from leader console-consumer-0f0146ce-4b8b-45d1-abca-9c8bdd1bf6dd for group groupe-test f
or generation 3. The group has 1 members, 0 of which are static. (org.apache.kafka.coordinator.group.
GroupMetadataManager)
[2026-10-01 10:41:38,066] INFO [GroupCoordinator id=1 topic=__consumer_offsets partition=25] [Group g
roupe-test] Member console-consumer-0f0146ce-4b8b-45d1-abca-9c8bdd1bf6dd has left group through expli
cit 'LeaveGroup' request; client reason: the consumer is being closed (org.apache.kafka.coordinator.g
roup.GroupMetadataManager)
[2026-10-01 10:41:38,067] INFO [GroupCoordinator id=1 topic=__consumer_offsets partition=25] Preparin
g to rebalance group groupe-test in state PreparingRebalance with old generation 3 (reason: explicit
'LeaveGroup' request for (console-consumer-0f0146ce-4b8b-45d1-abca-9c8bdd1bf6dd) members.). (org.apac
he.kafka.coordinator.group.GroupMetadataManager)
[2026-10-01 10:41:38,067] INFO [GroupCoordinator id=1 topic=__consumer_offsets partition=25] Group gr
oupe-test with generation 4 is now empty. (org.apache.kafka.coordinator.group.GroupMetadataManager)

```

Figure 25 — Journal du broker : entrée et sortie du membre de groupe-test

Commentaire. Le coordinateur de groupe enregistre l'arrivée du consumer (rebalance, generation 3) puis son départ (LeaveGroup) : le groupe devient vide mais ses offsets commités sont conservés.

5.5 Mission 15 — Alimenter RAW à partir du flux

Pourquoi pas « un événement = un fichier » ? À une vente par seconde, cela représenterait 86 400 objets par jour. Chaque objet coûte une requête S3, et la lecture devient dominée par le listage : c'est le **problème des petits fichiers**.

Stratégie retenue : le micro-batch temporel. Spark accumule les événements pendant 20 secondes (paramètre `--intervalle`) et écrit un seul fichier par lot. C'est un compromis entre fraîcheur des données (un délai court) et nombre de fichiers (un délai long). Un micro-batch vide ne produit aucun fichier.

5.6 Mission 16 — Organisation de l'historique streaming

```

raw/ventes_stream/date=2026-10-01/heure=11/batch_000012.jsonl    messages bruts + partition/offset
raw/_rejets_stream/date=2026-10-01/batch_000012.jsonl          événements hors schéma
processed/ventes_stream/date=2026-10-01/batch=000012/          ventes valides en Parquet
curated/streaming/ca_par_ville/ et curated/streaming/ca_par_categorie/

```

- découpage par date puis heure : retrouver ou rejouer une période ;
- nom = numéro de micro-batch : un batch rejoué réécrit les mêmes clés, sans doublon ;
- conservation du message brut avec sa partition et son offset : traçabilité jusqu'à Kafka ;
- préfixes distincts du batch (raw/ventes/) : aucun impact sur le pipeline existant.

5.7 Mission 17 — Spark Structured Streaming

`streaming_job.py` lit le topic avec `readStream.format("kafka")` et traite chaque micro-batch avec `foreachBatch` :

Étape	Identité	Traitement
RAW	ingestion-user	Écriture des messages bruts (JSONL) avec partition et offset

Étape	Identité	Traitement
Schéma	—	from_json avec un schéma explicite ; champs obligatoires présents, quantité entière > 0, prix > 0 ; non-conformes isolés dans _rejets_stream
PROCESSED	pipeline-user	Typage, montant, ville standardisée avec ville_standard() du batch, Parquet
CURATED	pipeline-user	Recalcul sur l'historique streaming avec agreger() du batch

Le **checkpoint** (dossier local checkpoints/) enregistre pour chaque micro-batch les offsets prévus puis la validation de fin. C'est lui qui permet la reprise. Les règles métier sont importées du pipeline batch, et non dupliquées.

Test — Premier micro-batch

Objectif

Vérifier la lecture de Kafka, l'écriture dans le Data Lake et le contrôle de schéma.

Action

Premier lancement de streaming_job.py (startingOffsets = earliest, aucun checkpoint).

Preuve

```
Streaming démarré : topic 'ventes', micro-batch toutes les 20 s, checkpoint ../checkpoints/ventes_stream
===== Micro-batch 0 – 11:06:59 – 213 message(s) : 210 valide(s), 3 rejeté(s) =====
Offsets traités : p0:0→30, p1:0→143, p2:0→37
RAW : raw/ventes_stream/date=2026-10-01/heure=11/batch_000000.jsonl
CA par ville (cumul streaming, 210 ventes) : Lyon 3481.2 | Nantes 2751.3 | Paris 2720.15 | ...
```

Extrait du terminal (sortie reproduite).

Observation

Les 213 messages présents depuis l'offset 0 sont traités en un seul micro-batch et un seul fichier RAW. Les 3 rejets sont les messages texte de la mission 11.

Interprétation

Le contrôle de schéma isole les données non conformes sans interrompre le flux. Ensuite, le traitement attend : un nouveau micro-batch n'est créé que si de nouveaux messages arrivent.

5.8 Mission 18 — Indicateur alimenté progressivement

Test — Évolution du CA par ville

Objectif

Montrer que l'indicateur évolue avec l'arrivée des événements.

Action

Caisse en continu (une vente par seconde), captures successives des micro-batches.

Preuve

```
+-----+-----+-----+-----+
|ville   |nb_ventes|quantite_totale|chiffre_affaires|
+-----+-----+-----+-----+
|Lyon    |42       |85             |3481.2           |
|Nantes  |31       |73             |2751.3           |
|Paris   |34       |73             |2720.15          |
|Lille   |35       |77             |2545.75          |
|Rennes  |32       |57             |2276.55          |
|Marseille|36       |61             |2130.25          |
+-----+-----+-----+-----+

-> s3a://tp2/processed/ventes_stream/date=2026-10-01/batch=000001/ (1 fichier(s) part + _SUCCESS)
-> s3a://tp2/curated/streaming/ca_par_ville/ (1 fichier(s) part + _SUCCESS)
-> s3a://tp2/curated/streaming/ca_par_categorie/ (1 fichier(s) part + _SUCCESS)

===== Micro-batch 1 - 11:12:00 - 19 message(s) : 19 valide(s), 0 rejeté(s) =====
Offsets traités : p0:31→34, p1:144→155, p2:38→40
RAW : raw/ventes_stream/date=2026-10-01/heure=11/batch_000001.jsonl
CA par ville (cumul streaming, 229 ventes) :
+-----+-----+-----+-----+
|ville   |nb_ventes|quantite_totale|chiffre_affaires|
+-----+-----+-----+-----+
|Lyon    |44       |88             |3620.9           |
|Nantes  |35       |81             |3004.05          |
|Rennes  |37       |70             |2882.15          |
|Paris   |36       |77             |2863.65          |
|Lille   |38       |82             |2714.25          |
|Marseille|39       |66             |2272.15          |
+-----+-----+-----+-----+

-> s3a://tp2/processed/ventes_stream/date=2026-10-01/batch=000002/ (1 fichier(s) part + _SUCCESS)
-> s3a://tp2/curated/streaming/ca_par_ville/ (1 fichier(s) part + _SUCCESS)
-> s3a://tp2/curated/streaming/ca_par_categorie/ (1 fichier(s) part + _SUCCESS)

===== Micro-batch 2 - 11:12:20 - 20 message(s) : 20 valide(s), 0 rejeté(s) =====
Offsets traités : p0:35→36, p1:156→170, p2:41→43
RAW : raw/ventes_stream/date=2026-10-01/heure=11/batch_000002.jsonl
CA par ville (cumul streaming, 249 ventes) :
+-----+-----+-----+-----+
|ville   |nb_ventes|quantite_totale|chiffre_affaires|
+-----+-----+-----+-----+
|Lyon    |47       |93             |3741.0           |
|Nantes  |37       |86             |3142.4           |
|Rennes  |40       |79             |3141.35          |
|Paris   |40       |83             |3005.05          |
|Lille   |41       |87             |2941.95          |
|Marseille|44       |77             |2795.75          |
+-----+-----+-----+-----+
```

Figure 26 — Micro-batches 1 et 2

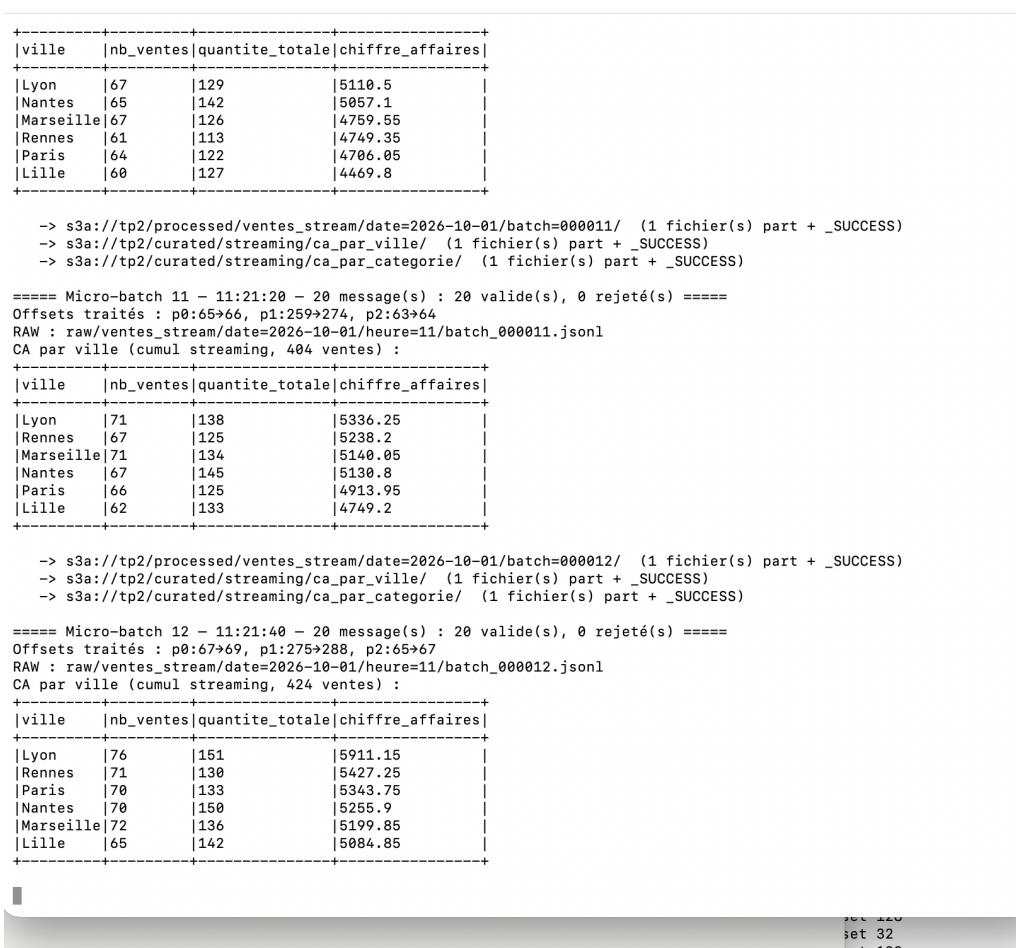


Figure 27 — Micro-batches 11 et 12

Observation

Environ 20 ventes par micro-batch. Le cumul passe de 210 à 229, 249, puis 404 et 424 ventes. Le CA de Lyon passe de 3 481,20 à 3 620,90, 3 741,00, puis 5 336,25 et 5 911,15.

Interprétation

Le résultat stocké dans curated/streaming/ est mis à jour à chaque micro-batch : l'indicateur suit l'activité avec une latence d'environ 20 secondes.

5.9 Mission 19 — Interruption et reprise

Test — Arrêt du streaming pendant la production

Objectif

Déterminer si les événements produits pendant un arrêt sont perdus, retraités ou repris depuis un point précis.

Action

Caisse en continu ; arrêt du streaming (Ctrl+C) pendant le traitement d'un micro-batch ; une minute d'attente ; redémarrage.

Preuve

```

==== Micro-batch 14 - 11:24:06 - 20 message(s) : 20 valide(s)
Offsets traités : p0:75→76, p1:301→315, p2:70→72 (cumul 463 ventes)
==== Micro-batch 15 - 11:24:20 - 118 message(s) : 118 valide(s)
Offsets traités : p0:77→91, p1:316→394, p2:73→96 (cumul 581 ventes)
==== Micro-batch 16 - 11:24:40 - 19 message(s) : 19 valide(s)
Offsets traités : p0:92→92, p1:395→409, p2:97→99 (cumul 600 ventes)

```

Observation

Le batch 14, interrompu par l'arrêt, est rejoué avec exactement les mêmes offsets (301→315). Le batch 15 contient 118 messages, c'est-à-dire toutes les ventes produites pendant l'arrêt. Le batch 16 revient au rythme normal. Les offsets s'enchaînent sans trou (315→316, 394→395) et le cumul augmente exactement du nombre de messages traités.

Interprétation

Aucun événement perdu : Kafka les a conservés. **Aucun doublon** : le batch non validé est rejoué à l'identique et réécrit les mêmes clés. La reprise part d'un **point précis**, le checkpoint : Spark y note les offsets prévus d'un batch avant de le traiter, puis sa validation. Au redémarrage, il rejoue le dernier batch non validé puis lit la suite. À la différence de la mission 14 (offsets mémorisés par Kafka pour un consumer group), c'est ici Spark qui détient la position, ce qui lui permet d'aligner lecture et écriture.

5.10 Mission 20 — Cohabitation batch et streaming

Test — Batch pendant le streaming

Objectif

Vérifier que les traitements batch de la phase 2 restent opérationnels pendant le streaming.

Action

Caisse et streaming en marche ; dépôt de ventes_0310.csv (V301, V302) dans incoming/ et run_batch.py.

Preuve

```

tp2 -- -zsh -- 116x50
...ktop/datalake/tp2 -- java - Python src/streaming/streaming_job.py
~/Desktop/data

-> s3a://tp2/processed/ventes/ (1 fichier(s) part + _SUCCESS)
clients : 11 ligne(s) après nettoyage
-> s3a://tp2/processed/clients/ (1 fichier(s) part + _SUCCESS)
produits : 11 ligne(s) après nettoyage
-> s3a://tp2/processed/produits/ (1 fichier(s) part + _SUCCESS)
magasins : 6 ligne(s) après nettoyage
-> s3a://tp2/processed/magasins/ (1 fichier(s) part + _SUCCESS)

[3/4] Enrichissement des ventes (lecture depuis PROCESSED)
ventes exploitables : 87 | écartées : 5
+-----+-----+-----+-----+-----+
|id_vente|id_produit|quantite|prix_unitaire|anomalies|
+-----+-----+-----+-----+-----+
|V012|P010|-2|21.5|quantite negative|
|V053|P004|0|59.0|quantite nulle|
|V208|P007|0|39.9|quantite nulle|
|V032|P999|2|NULL|produit inconnu|
|V018|P003|0|49.9|quantite absente|
+-----+-----+-----+-----+-----+

-> s3a://tp2/processed/ventes_enrichies/ (1 fichier(s) part + _SUCCESS)

[4/4] PROCESSED -> CURATED (agrégations)
-> s3a://tp2/curated/ca_par_ville/ (1 fichier(s) part + _SUCCESS)
-> s3a://tp2/curated/ca_par_categorie/ (1 fichier(s) part + _SUCCESS)

=== Contrôle des résultats CURATED (relus depuis MinIO) ===
+-----+-----+-----+-----+
|ville|nb_ventes|quantite_totale|chiffre_affaires|
+-----+-----+-----+-----+
|Paris|19|62|2268.0|
|Lille|17|65|2215.4|
|Lyon|13|52|2066.4|
|Rennes|13|42|1421.85|
|Marseille|14|43|1409.2|
|Nantes|10|28|1286.6|
|Inconnue|1|5|199.5|
+-----+-----+-----+-----+

+-----+-----+-----+-----+
|categorie|nb_ventes|quantite_totale|chiffre_affaires|
+-----+-----+-----+-----+
|Audio|24|70|4037.1|
|Informatique|32|115|3737.9|
|Accessoires|20|74|2392.4|
|Maison|11|38|699.55|
+-----+-----+-----+-----+

Pipeline terminé avec succès. (raw/images_produits/ conservées dans RAW uniquement)
(.venv) rekiknour@KHALESSI tp2 %

tp2 -- Python src/streaming/producer.py --rythme 1 -- 202x56
#344 S-E982798361 Nantes Informatique x2 -> partition 0, offset 103
#345 S-59588BF9CEC Rennes Informatique x3 -> partition 1, offset 470
#346 S-7F9CE450EE Rennes Maison x2 -> partition 1, offset 471
#347 S-384272D7C6 Lyon Accessoires x3 -> partition 1, offset 472
#348 S-C146276813 Lyon Accessoires x3 -> partition 1, offset 473
#349 S-R162579C42 Nantes Audio x1 -> partition 0, offset 104
#350 S-E9325C85FC Marseille Audio x1 -> partition 1, offset 474
#351 S-D14958C835 Lyon Informatique x3 -> partition 1, offset 475
#352 S-E191499D42 Nantes Informatique x2 -> partition 0, offset 105
#353 S-1075A23882 Paris Audio x1 -> partition 1, offset 476
#354 S-D7392EE910 Paris Informatique x4 -> partition 1, offset 477
#355 S-9D1070DC29 Marseille Informatique x3 -> partition 1, offset 478
#356 S-3983A372D5 Lille Audio x2 -> partition 2, offset 107
#357 S-99F5FB57EC Rennes Accessoires x2 -> partition 1, offset 479
#358 S-C9338F9648 Lyon Accessoires x1 -> partition 1, offset 480
#359 S-5729256277 Lille Informatique x1 -> partition 2, offset 108
#360 S-7878C685D5 Lyon Informatique x1 -> partition 1, offset 481
#361 S-353E684964 Nantes Audio x3 -> partition 0, offset 106
#362 S-3CCE2E4544 Lyon Audio x1 -> partition 1, offset 482
#363 S-E5CE520DBF Lyon Informatique x3 -> partition 1, offset 483
#364 S-6289E50C63 Lille Maison x2 -> partition 2, offset 109
#365 S-45666CE0EB Rennes Informatique x2 -> partition 1, offset 484
#366 S-S09F98BE36 Nantes Informatique x1 -> partition 0, offset 107
#367 S-1C06745714 Lyon Audio x1 -> partition 1, offset 485
#368 S-80DF47F683C Rennes Accessoires x1 -> partition 1, offset 486
#369 S-D3F3B2C221 Paris Accessoires x1 -> partition 1, offset 487
#370 S-CD99E5E871 Marseille Maison x5 -> partition 1, offset 488
#371 S-5360B20CE3 Nantes Informatique x2 -> partition 0, offset 108
#372 S-0105FA22D3 Rennes Informatique x1 -> partition 1, offset 489
#373 S-C55EDC45F5 Paris Accessoires x4 -> partition 1, offset 490
#374 S-33F7AC5979 Lille Audio x1 -> partition 2, offset 110
#375 S-AC887EBE82 Marseille Accessoires x1 -> partition 1, offset 491
#376 S-C2A1CDA462 Lyon Audio x2 -> partition 1, offset 492
#377 S-9872727BF5 Rennes Informatique x1 -> partition 1, offset 493
#378 S-9078EBCE21 Paris Audio x1 -> partition 1, offset 494
#379 S-E492B2DDFC Paris Accessoires x2 -> partition 1, offset 495
#380 S-93C5E88B60 Marseille Accessoires x3 -> partition 1, offset 496
#381 S-93364AF530 Rennes Informatique x4 -> partition 1, offset 497
#382 S-61D15B4848 Lille Accessoires x3 -> partition 2, offset 111
#383 S-A3F8A2DE03 Marseille Audio x1 -> partition 1, offset 498
#384 S-59EFB655A2 Nantes Audio x3 -> partition 0, offset 109
#385 S-15D2A8B874 Lille Audio x2 -> partition 2, offset 112
#386 S-920C367A99 Lille Audio x3 -> partition 2, offset 113
#387 S-2F186D4E3C Paris Accessoires x1 -> partition 1, offset 499
#388 S-3742D50728 Paris Informatique x1 -> partition 1, offset 500
#389 S-2E3C998DDE Lyon Audio x1 -> partition 1, offset 501
#390 S-A9AD4B604D Rennes Informatique x4 -> partition 1, offset 502
#391 S-427208D559 Paris Audio x4 -> partition 1, offset 503
#392 S-9254F15FF2 Lille Informatique x2 -> partition 2, offset 114
#393 S-786D9F43B5 Lyon Audio x3 -> partition 1, offset 504
#394 S-50F5C0D1F4 Marseille Maison x1 -> partition 1, offset 505
#395 S-48C0F906CF Paris Informatique x3 -> partition 1, offset 506
#396 S-18376FF303 Rennes Accessoires x1 -> partition 1, offset 507

```

Figure 28 — Pipeline batch terminé pendant que la caisse émet

Observation

Ingestion SUCCESS ; 87 ventes exploitables ; Paris 2 268,00 et Lille 2 215,40. Le streaming continue ses micro-batches. Spark signale que le port 4040 est occupé et utilise le port 4041.

Interprétation

Deux applications Spark tournent simultanément sans conflit. Les deux chaînes partagent le stockage, les identités et les règles métier, mais écrivent dans des préfixes distincts (raw/ventes/ et raw/ventes_stream/, curated/ca_par_* et curated/streaming/).

5.11 Mission 21 — Incidents

Incident 1 — Événements hors schéma

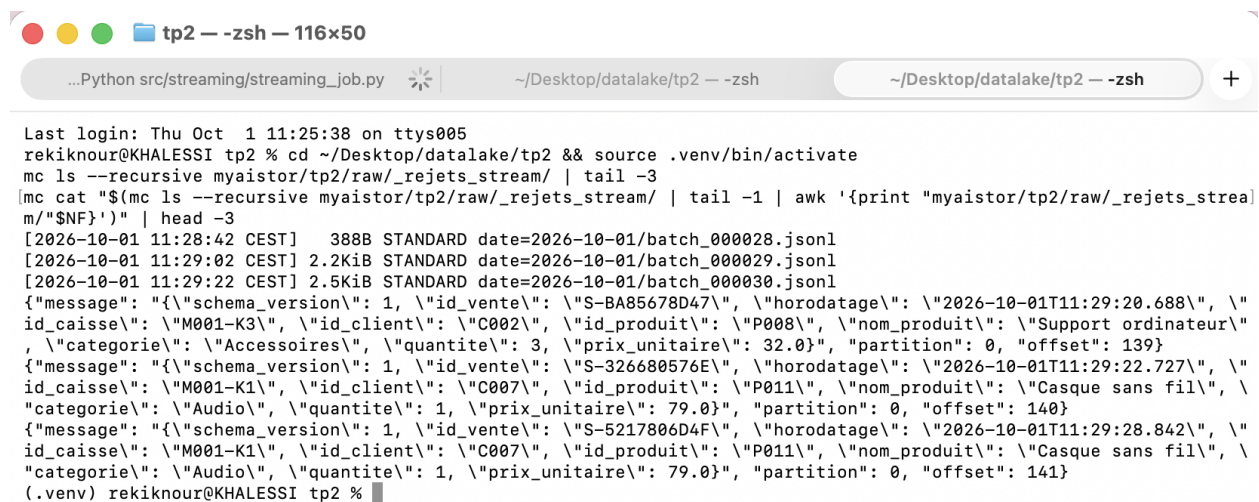
Objectif

Vérifier qu'un événement non conforme ne bloque pas le flux et ne fausse pas les indicateurs.

Action

Producer relancé avec --taux-invalides 0.3 (quantité en texte, prix négatif, magasin et ville absents) ; **détection** dans les compteurs des micro-batches ; **diagnostic** par lecture de _rejets_stream ; **correction** : retour au producer normal.

Preuve



```
tp2 — -zsh — 116x50
...Python src/streaming/streaming_job.py
~/Desktop/datalake/tp2 — -zsh
~/Desktop/datalake/tp2 — -zsh
Last login: Thu Oct 1 11:25:38 on ttys005
rekiknour@KHALESSI tp2 % cd ~/Desktop/datalake/tp2 && source .venv/bin/activate
mc ls --recursive myaistor/tp2/raw/_rejets_stream/ | tail -3
[mc cat "$(mc ls --recursive myaistor/tp2/raw/_rejets_stream/ | tail -1 | awk '{print "myaistor/tp2/raw/_rejets_strea
m/"$NF}')" | head -3
[2026-10-01 11:28:42 CEST] 388B STANDARD date=2026-10-01/batch_000028.jsonl
[2026-10-01 11:29:02 CEST] 2.2KiB STANDARD date=2026-10-01/batch_000029.jsonl
[2026-10-01 11:29:22 CEST] 2.5KiB STANDARD date=2026-10-01/batch_000030.jsonl
{"message": "{\"schema_version\": 1, \"id_vente\": \"S-BA85678047\", \"horodatage\": \"2026-10-01T11:29:20.688\", \"
id_caisse\": \"M001-K3\", \"id_client\": \"C002\", \"id_produit\": \"P008\", \"nom_produit\": \"Support ordinateur\",
, \"categorie\": \"Accessoires\", \"quantite\": 3, \"prix_unitaire\": 32.0\", \"partition\": 0, \"offset\": 139}
{"message": "{\"schema_version\": 1, \"id_vente\": \"S-326680576E\", \"horodatage\": \"2026-10-01T11:29:22.727\", \"
id_caisse\": \"M001-K1\", \"id_client\": \"C007\", \"id_produit\": \"P011\", \"nom_produit\": \"Casque sans fil\", \"
categorie\": \"Audio\", \"quantite\": 1, \"prix_unitaire\": 79.0\", \"partition\": 0, \"offset\": 140}
{"message": "{\"schema_version\": 1, \"id_vente\": \"S-5217806D4F\", \"horodatage\": \"2026-10-01T11:29:28.842\", \"
id_caisse\": \"M001-K1\", \"id_client\": \"C007\", \"id_produit\": \"P011\", \"nom_produit\": \"Casque sans fil\", \"
categorie\": \"Audio\", \"quantite\": 1, \"prix_unitaire\": 79.0\", \"partition\": 0, \"offset\": 141}
(.venv) rekiknour@KHALESSI tp2 %
```

Figure 29 — Événements rejetés conservés dans _rejets_stream

Observation

Les micro-batches affichent des rejets. Les événements sans id_magasin ni ville sont isolés avec leur partition et leur offset (partition 0, offsets 139 à 141).

Interprétation

Le flux continue. Les rejets n'entrent ni dans PROCESSED ni dans CURATED ; leur partition et leur offset permettent de les retrouver dans Kafka et de les rejouer une fois la source corrigée.

Incident 2 — MinIO temporairement inaccessible

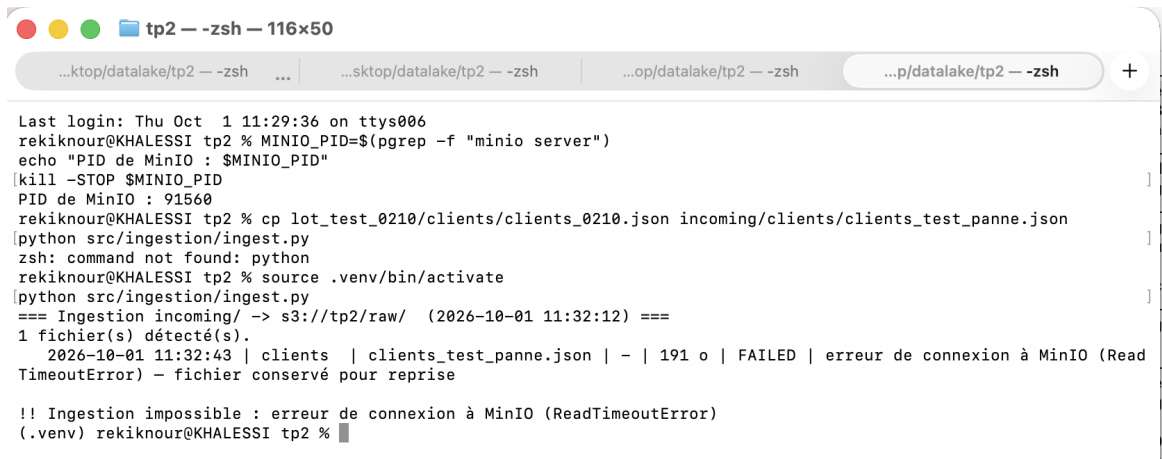
Objectif

Observer le comportement de l'ingestion et du streaming lorsque le stockage ne répond plus.

Action

MinIO gelé (kill -STOP) ; **détection** : ingestion et streaming ; **diagnostic** ; **correction** : kill -CONT ; **reprise** : nouvelle ingestion et redémarrage du streaming.

Preuve



```

tp2 — -zsh — 116x50
...ktop/datalake/tp2 — -zsh ...
...sktop/datalake/tp2 — -zsh
...op/datalake/tp2 — -zsh
...p/datalake/tp2 — -zsh +

Last login: Thu Oct  1 11:29:36 on ttys006
rekiknour@KHALESSI tp2 % MINIO_PID=$(pgrep -f "minio server")
echo "PID de MinIO : $MINIO_PID"
kill -STOP $MINIO_PID
PID de MinIO : 91560
rekiknour@KHALESSI tp2 % cp lot_test_0210/clients/clients_0210.json incoming/clients/clients_test_panne.json
python src/ingestion/ingest.py
zsh: command not found: python
rekiknour@KHALESSI tp2 % source .venv/bin/activate
python src/ingestion/ingest.py
=== Ingestion incoming/ -> s3://tp2/raw/ (2026-10-01 11:32:12) ===
1 fichier(s) détecté(s).
2026-10-01 11:32:43 | clients | clients_test_panne.json | - | 191 o | FAILED | erreur de connexion à MinIO (Read
TimeoutError) - fichier conservé pour reprise

!! Ingestion impossible : erreur de connexion à MinIO (ReadTimeoutError)
(.venv) rekiknour@KHALESSI tp2 %

```

Figure 30 — Ingestion pendant la panne : FAILED

```

botocore.exceptions.ReadTimeoutError: Read timeout on endpoint URL:
"http://127.0.0.1:9000/tp2/raw/ventes_stream/.../batch_000036.jsonl"
StreamingQueryException: [STREAM_FAILED] ... [FOREACH_BATCH_USER_FUNCTION_ERROR]
Current Committed Offsets: {"ventes":{"0":155,"1":651,"2":152}}
Current Available Offsets: {"ventes":{"0":157,"1":667,"2":154}}

```

Extrait du terminal : arrêt du streaming (sortie reproduite).

```

[ (.venv) rekiknour@KHALESSI tp2 % python src/ingestion/ingest.py ]
=== Ingestion incoming/ -> s3://tp2/raw/ (2026-10-01 11:33:47) ===
1 fichier(s) détecté(s).
2026-10-01 11:33:47 | clients | clients_test_panne.json | raw/clients/ingest_date=2026-10-01/clients_0210.json |
191 o | DUPLICATE | contenu identique déjà ingéré le 2026-10-01 10:21:06 (clients_0210.json)

Bilan : 0 ingéré(s), 1 doublon(s) ignoré(s), 0 rejeté(s), 0 en échec (à reprendre).
Journal : /Users/rekiknour/Desktop/datalake/tp2/logs/ingestion.log
[ (.venv) rekiknour@KHALESSI tp2 % python src/ingestion/ingest.py ]
=== Ingestion incoming/ -> s3://tp2/raw/ (2026-10-01 11:35:02) ===
Aucun nouveau fichier dans incoming/.
[ (.venv) rekiknour@KHALESSI tp2 % ]

```

Figure 31 — Reprise de l'ingestion

```

+-----+-----+-----+-----+
|ville|nb_ventes|quantite_totale|chiffre_affaires|
+-----+-----+-----+-----+
|Lyon|172|350|14183.35|
|Marseille|176|341|12723.3|
|Paris|153|296|11452.15|
|Rennes|148|302|11396.1|
|Nantes|147|308|11229.95|
|Lille|143|296|10755.1|
+-----+-----+-----+-----+

-> s3a://tp2/processed/ventes_stream/date=2026-10-01/batch=000037/ (1 fichier(s) part + _SUCCESS)
-> s3a://tp2/curated/streaming/ca_par_ville/ (1 fichier(s) part + _SUCCESS)
-> s3a://tp2/curated/streaming/ca_par_categorie/ (1 fichier(s) part + _SUCCESS)

===== Micro-batch 37 - 11:35:12 - 228 message(s) : 228 valide(s), 0 rejeté(s) =====
Offsets traités : p0:157→197, p1:667→813, p2:154→193
RAW : raw/ventes_stream/date=2026-10-01/heure=11/batch_000037.jsonl
CA par ville (cumul streaming, 1167 ventes) :
+-----+-----+-----+-----+
|ville|nb_ventes|quantite_totale|chiffre_affaires|
+-----+-----+-----+-----+
|Lyon|217|451|18027.05|
|Marseille|217|435|16378.25|
|Nantes|188|391|14616.15|
|Lille|183|382|14200.75|
|Paris|184|364|13902.3|
|Rennes|178|361|13719.45|
+-----+-----+-----+-----+

-> s3a://tp2/processed/ventes_stream/date=2026-10-01/batch=000038/ (1 fichier(s) part + _SUCCESS)
-> s3a://tp2/curated/streaming/ca_par_ville/ (1 fichier(s) part + _SUCCESS)
-> s3a://tp2/curated/streaming/ca_par_categorie/ (1 fichier(s) part + _SUCCESS)

===== Micro-batch 38 - 11:35:20 - 8 message(s) : 8 valide(s), 0 rejeté(s) =====
Offsets traités : p1:814→821
RAW : raw/ventes_stream/date=2026-10-01/heure=11/batch_000038.jsonl
CA par ville (cumul streaming, 1175 ventes) :
+-----+-----+-----+-----+
|ville|nb_ventes|quantite_totale|chiffre_affaires|
+-----+-----+-----+-----+
|Lyon|217|451|18027.05|
|Marseille|221|447|16731.35|
|Nantes|188|391|14616.15|
|Paris|187|372|14256.7|
|Lille|183|382|14200.75|
|Rennes|179|365|13817.45|
+-----+-----+-----+-----+

[Stage 119:> (0 + 1) / 1]

```

Figure 32 — Reprise du streaming

Observation

L'ingestion échoue après environ 30 s (ReadTimeoutError) et conserve le fichier. Le streaming s'arrête sur le micro-batch 36 et indique les offsets validés (155 / 651 / 152) et disponibles. Après réveil de MinIO, le batch 36 est rejoué depuis les offsets validés, puis le batch 37 rattrape 228 ventes accumulées dans Kafka.

Interprétation

Face à une panne de stockage, le système s'arrête proprement au lieu d'écrire des données partielles. Kafka sert de tampon ; le checkpoint indique précisément où reprendre ; les noms de fichiers fixes empêchent les doublons. Aucune vente n'est perdue.

Incident 3 — Kafka temporairement arrêté

Objectif

Observer le comportement du producer et du streaming quand le broker est arrêté.

Action

Arrêt du broker (Ctrl+C) ; **détection** côté producer et streaming ; **correction** : kafka-server-start ; **reprise** automatique du producer et du streaming.

Preuve

```

#370 S-14BC7985CA Marseille Audio x4 -> partition 1, offset 874
#371 S-3A3B8FDC73 Rennes Accessoires x1 -> partition 1, offset 875
!!! échec d'envoi de S-EF185C0BE9 : KafkaTimeoutError KafkaTimeoutError: Timeout after waiting for 10 secs.
!!! échec d'envoi de S-1EDA0FE0B3 : KafkaTimeoutError KafkaTimeoutError: Timeout after waiting for 10 secs.

```

Figure 33 — Producer pendant l'arrêt de Kafka

Observation

Le producer affiche des échecs `KafkaTimeoutError` toutes les 10 secondes ; le streaming ne produit plus de micro-batch. Après redémarrage, le producer se reconnecte et les offsets reprennent à la suite (sans repartir de 0) ; le streaming reprend depuis son checkpoint.

Interprétation

Kafka persiste ses messages sur disque : rien de ce qu'il avait reçu n'est perdu. **Limite** : les ventes générées pendant la panne sont perdues côté caisse, car notre producer ne les met pas en attente. En production, la caisse les conserverait dans un tampon local et les renverrait au retour du broker.

6. Architecture V3 — Batch + Streaming

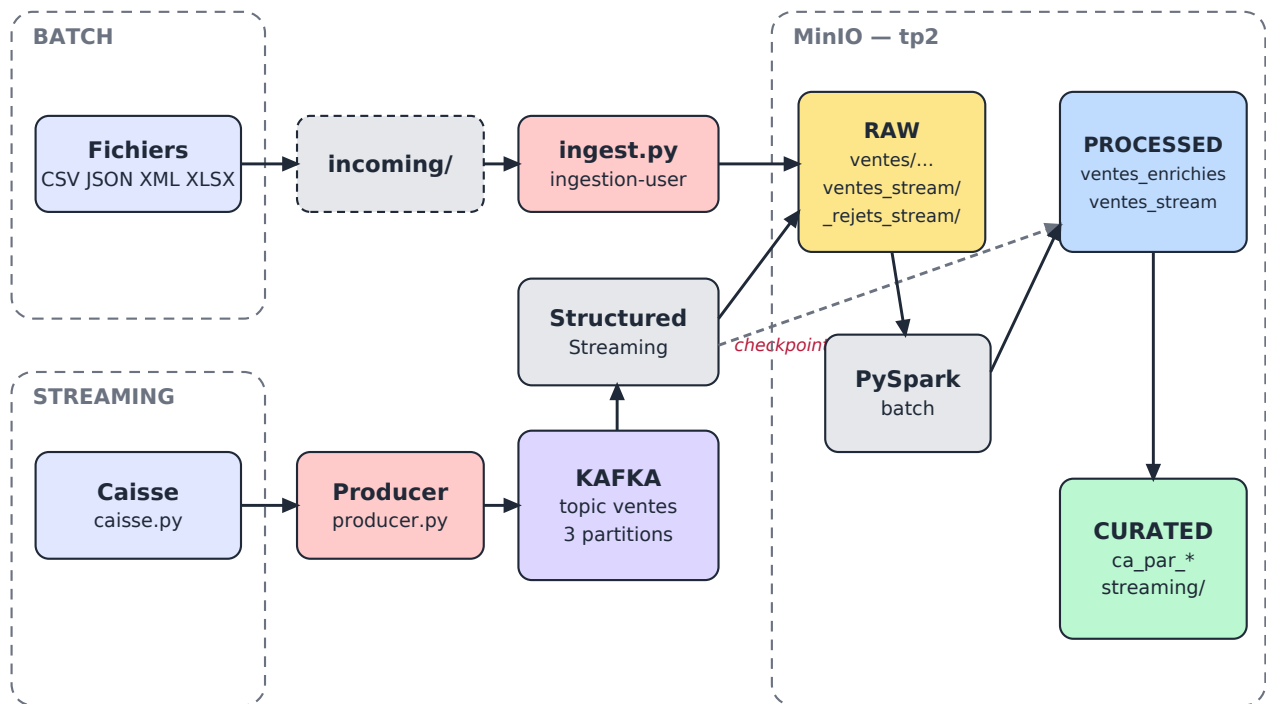


Figure 34 — Architecture V3 — Batch + Streaming

Lecture du schéma. Deux chemins d'alimentation coexistent : les fichiers passent par incoming/ et l'ingestion Python ; les événements passent par le producer, Kafka et Spark Structured Streaming. Les deux aboutissent dans les mêmes zones RAW, PROCESSED et CURATED, sous des préfixes distincts, avec les mêmes identités et les mêmes règles métier.

7. Comparaison V1 / V2 / V3

	V1 — Socle	V2 — Ingestion automatisée	V3 — Batch + Streaming
Alimentation	Manuelle (mc cp)	incoming/ + ingest.py	Batch + événements Kafka
Latence	Celle de l'opérateur	À chaque exécution de run_batch.py	≈ 20 s (micro-batch)
Sécurité	Admin + pipeline-tp2	ingestion-user / pipeline-user	Idem, appliqué aussi au streaming
Contrôles	À la transformation	+ contrôles à l'arrivée	+ schéma des événements
Doublons	Dans les données	+ hash des fichiers	+ rejeu par numéro de micro-batch
Traçabilité	Aucune	Journal, manifest, archives	+ partition / offset Kafka
Reprise	Relancer à la main	Fichiers conservés dans incoming/	Checkpoint + rétention Kafka

Évolution des responsabilités

Composant	Évolution
MinIO	Stockage passif en V1 ; devient le point de convergence du batch et du streaming en V3, avec des préfixes dédiés et un manifest d'ingestion.

Composant	Évolution
Identités IAM	Une identité de traitement en V1 ; séparation ingestion / traitement en V2, réutilisée telle quelle par le streaming en V3.
Ingestion	Inexistante en V1 (geste humain) ; programme contrôlé et traçable en V2 ; double canal en V3.
PySpark	Seul composant actif en V1 ; réutilisé sans réécriture en V2 ; ses règles sont partagées avec le streaming en V3.
Kafka	Absent en V1 et V2 ; en V3, il découple la source temps réel du traitement et absorbe les indisponibilités.

8. Difficultés rencontrées et solutions

Difficulté	Solution
Écriture Spark impossible sans droit de suppression (committer_temporary, mode overwrite)	Écriture locale par Spark puis PutObject sous des clés fixes, conservée depuis la phase 1 et réutilisée par le streaming
Secret Key mal recopiée (SignatureDoesNotMatch, 8/10)	Génération d'une nouvelle clé, contrôle de sa longueur, suppression de la clé inutile
Fichiers livrés absents du projet (script introuvable, dossier « src 2 »)	Vérification systématique avec ls avant exécution
Messages Kafka non envoyés (saisie sans validation)	Envoi par printf avec retours à la ligne explicites
Erreurs Java à l'arrêt du streaming pendant un batch	Comportement attendu : le batch non validé est rejoué au redémarrage, sans doublon
Simuler une panne MinIO sans perdre la licence AIStor	Gel du processus (kill -STOP / -CONT) ou port fermé dans la configuration
Fichiers Excel non lus par Spark	Lecture par l'API S3 puis pandas/openpyxl, conversion en DataFrame

9. Conclusion technique

La plateforme a évolué sans être reconstruite. La phase 2 a supprimé toute intervention manuelle dans MinIO en ajoutant une ingestion contrôlée, traçable et rejouable, exécutée par une identité dédiée. La phase 3 a ajouté un canal temps réel avec Kafka et Spark Structured Streaming, dont les tests d'interruption et d'incident montrent qu'aucun événement reçu par Kafka n'est perdu ni compté deux fois, grâce au checkpoint et à des écritures idempotentes.

Limites actuelles

- les lancements sont manuels : pas d'ordonnanceur (cron, Airflow) ni de surveillance continue ;
- un seul broker Kafka, sans réplication : la panne du broker interrompt le flux ;
- la caisse perd les ventes émises pendant une panne de Kafka (pas de tampon local) ;
- le CURATED streaming est recalculé entièrement à chaque micro-batch, ce qui ne passera pas à l'échelle ;
- l'écriture par clés fixes suppose un nombre de fichiers constant ; le checkpoint est local au poste ;
- batch et streaming produisent des indicateurs séparés, non encore consolidés.

Évolutions possibles

- orchestration avec Airflow et supervision avec Prometheus et Grafana (lag Kafka, échecs d'ingestion) ;
- cluster Kafka à trois brokers avec réplication ; tampon local côté caisse ;
- agrégations incrémentales (état Spark ou fenêtres temporelles) au lieu du recalcul complet ;
- format de table transactionnel (Delta Lake ou Apache Iceberg) pour des écritures atomiques et la fusion batch / streaming ;
- registre de schémas (Avro, Schema Registry) pour versionner le contrat des événements ;
- checkpoint stocké dans MinIO avec une identité dédiée au streaming.